

Case Study

Phase 1

Deliverables

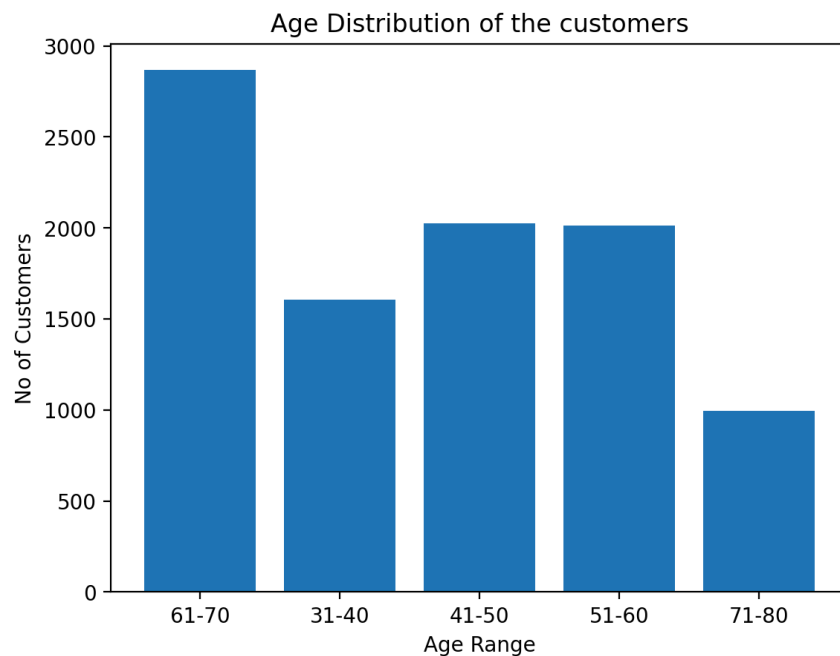
Develop a SQL solution/code-set of report-ready views based on the 3 scenarios from the database concepts section and the additional requirements provided above

Scenario 1: Viewing customer segments based on demographic information

- What is the **age and gender distribution of investment customers?**

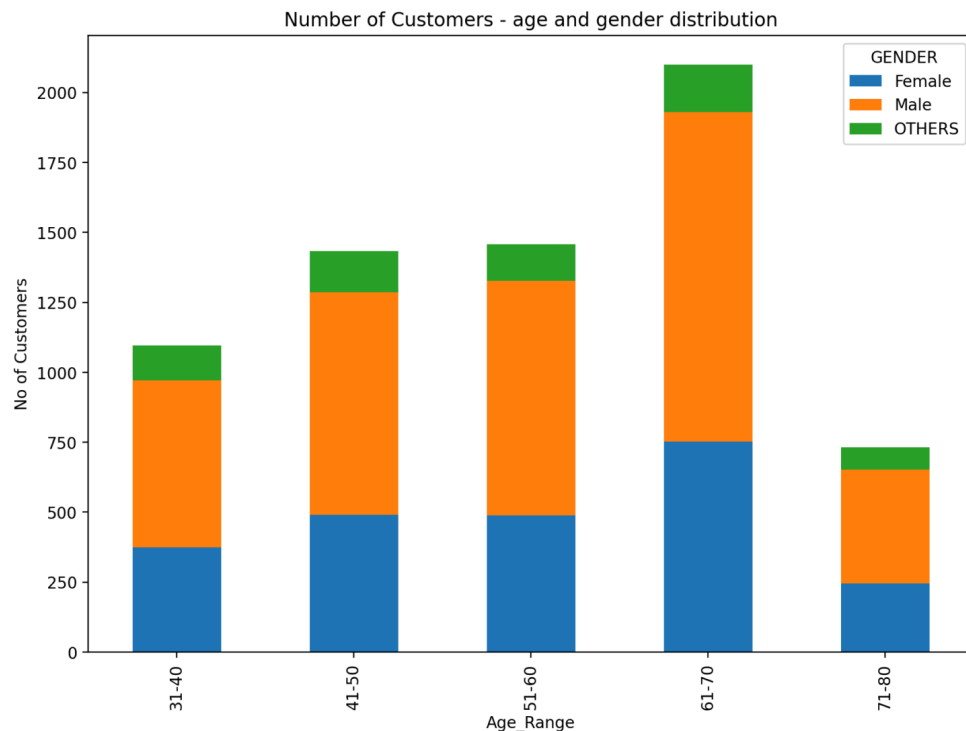
create or replace view

```
CASESTUDY_WORKING.PUBLIC.V_CUSTOMERS_AGE_DISTRIBUTION  
(AGE_RANGE, NO_OF_CUSTOMERS) as select get_age_range(age) as age_range  
,count(customer_id) as no_of_customers from eligible_customers group by  
get_age_range(age);
```



Age and Gender Distribution:

create or replace view v_age_gender_dist as select count(customer_id) as NO_OF_CUSTOMERS,coalesce(gender,'OTHERS') as gender,get_age_range(age) as AGE_RANGE from v_eligible_customers group by gender,get_age_range(age);

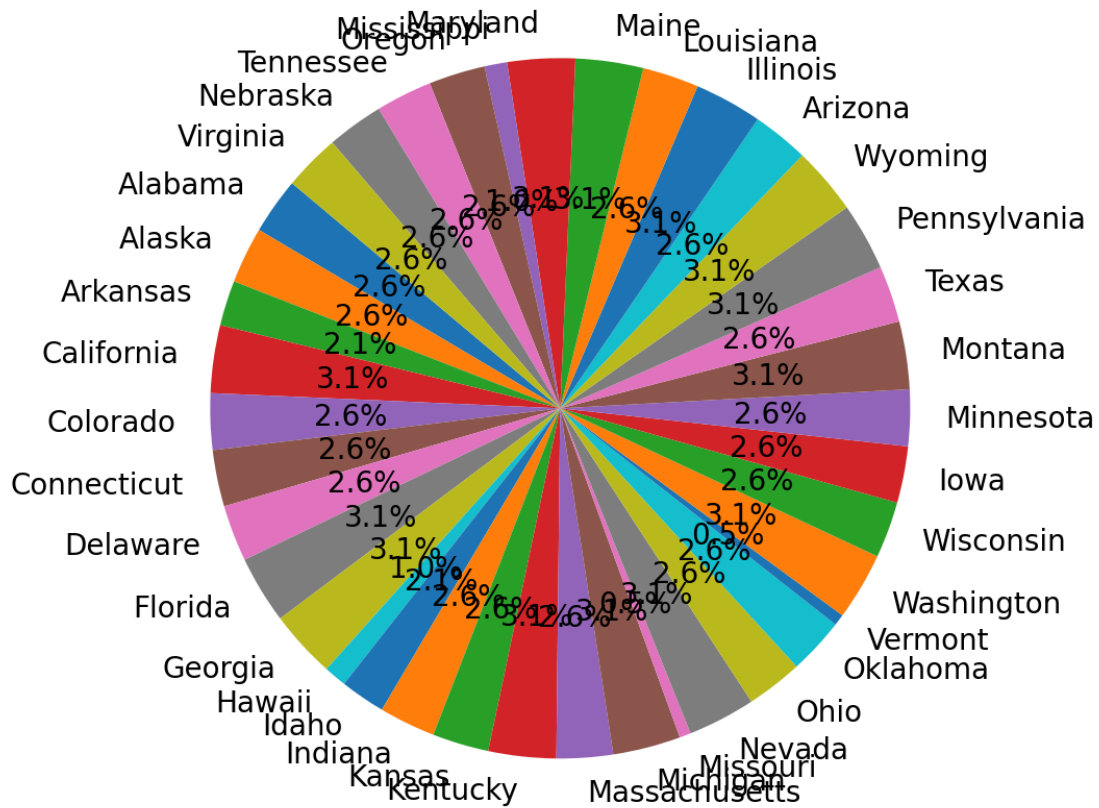


Here we can see that the customers are highly distributed in the age range of 61-70 with more no of males.

- **What is the regional distribution of investment customers?**

create or replace view

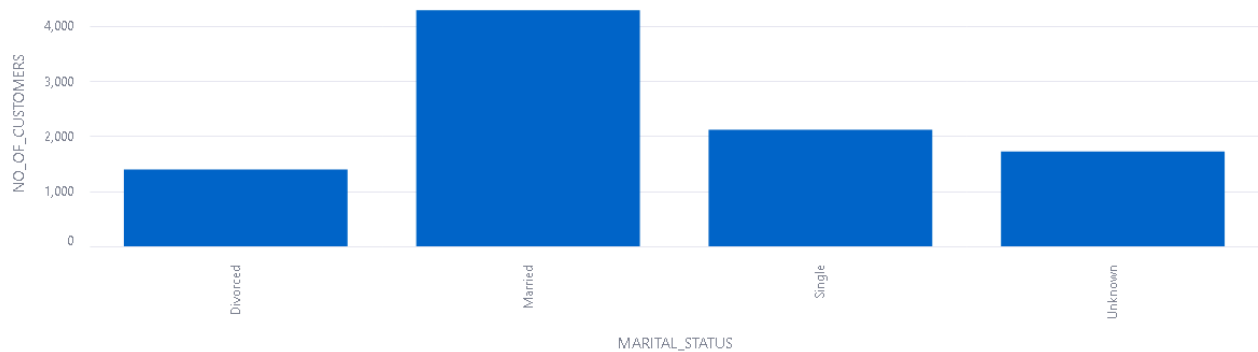
CASESTUDY_WORKING.PUBLIC.V_REGIONAL_DISTRIBUTION(STATE, CITY, NO_OF_CUSTOMERS) as select state,city ,count(customer_id) as no_of_customers from eligible_customers group by city,state order by state;



- **Are customers married, divorced, or single?**

create or replace view

```
CASESTUDY_WORKING.PUBLIC.V_MARITAL_STATUS(
MARITAL_STATUS, NO_OF_CUSTOMERS) as select Coalesce(
marital_status,'Unknown') as marital_status,count(customer_id) as
no_of_customers from eligible_customers group by marital_status;
```



- **Is there a relationship between a customer's available assets and number of dependents?**

Considering only the customers based on the data Risk Questionnaire analysis we have created a view that includes the customers who have answer data for at least 6 questions.

create or replace view

```
CASESTUDY_WORKING.PUBLIC.ELIGIBLE_CUSTOMERS(CUSTOMER_ID, CONTACT_LAST_NAME, CONTACT_FIRST_NAME, STREET, CITY, STATE, ZIP, COUNTRY, AGE, MARITAL_STATUS, GENDER, NUMBER_OF_DEPENDENTS, CID, "COUNT(CA.QUESTION_ID)") as (
```

```
with valid_customer_ids as (select customer_id as cid, count(ca.question_id)
from customer_answers ca join answers a on ca.answer_id=a.answer_id and
ca.question_id=a.question_id group by customer_id having
count(ca.question_id)>=6)
```

```
select * from customers c inner join valid_customer_ids vc on
c.customer_id=vc.cid);
```

- ***Do not include individuals who have dependents but available assets of less than \$100,000***

As they have asked not to include the customers who have available assets less than \$100,000 we have filtered out the customers based on this condition.

create or replace view

```
CASESTUDY_WORKING.PUBLIC.V_ELIGIBLE_CUSTOMERS(  
CUSTOMER_ID,CONTACT_LAST_NAME, CONTACT_FIRST_NAME,  
STREET, CITY, STATE, ZIP, COUNTRY, AGE, MARITAL_STATUS, GENDER,  
NUMBER_OF_DEPENDENTS, CID, "COUNT(CA.QUESTION_ID)"
```

) as

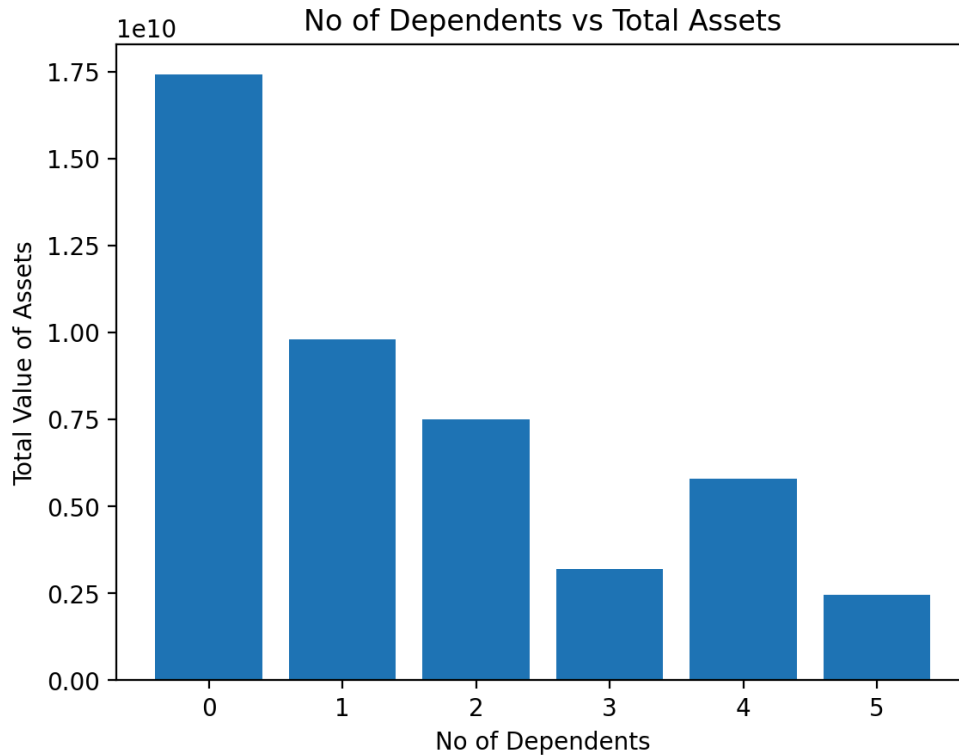
```
select * from eligible_customers where customer_id not in (
```

```
select ec.customer_id from customer_assets ca join eligible_customers ec on  
ca.customer_id=ec.customer_id where total < 100000 order by ec.customer_id);
```

The view is also created for the customers assets and the no of dependents.

create or replace view

```
CASESTUDY_WORKING.PUBLIC.V_ELIGIBLE_CUSTOMERS_DEPENDEN  
TS( CUSTOMER_ID, NUMBER_OF_DEPENDENTS, TOTAL) as select  
ec.customer_id, ec.number_of_dependents, cs.total from v_eligible_customers ec  
join customer_assets cs on cs.customer_id=ec.customer_id ;
```



The customers with less dependents have more total value of assets.

Creating customer segments based on gender and age:

There are 5 age ranges (31-40,41-50,51-60,61-70,71-80) and 2 genders are considered (Male and female), so totally ten segments will be created based on that.

V_segments

- Contains range of assets , maximum assets , minimum assets and average assets for each segment

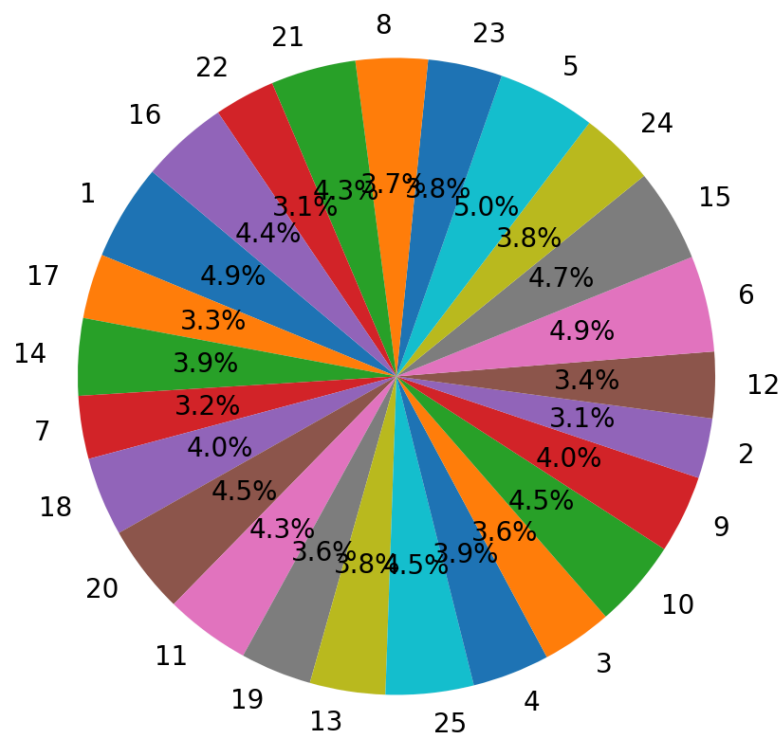
Customers Segmentation based on risk profile ID and Engagement Type ID:

The Risk Profiles that are present in the data is ranging from 1 to 5, similarly there are 5 types of engagement ids from 1 to 5. Here we are considering grouping the customers based on the risk profiles and the engagement types. There will be total of 25 segments(5*5), eg: risk profile id =1 and engagement id =1,

risk_profile_id=1 and engagement id=2, etc.

1	1	1
2	2	1
3	3	1
4	4	1
5	5	1
6	1	2
7	2	2
8	3	2
9	4	2
10	5	2
11	1	3
12	2	3
13	3	3

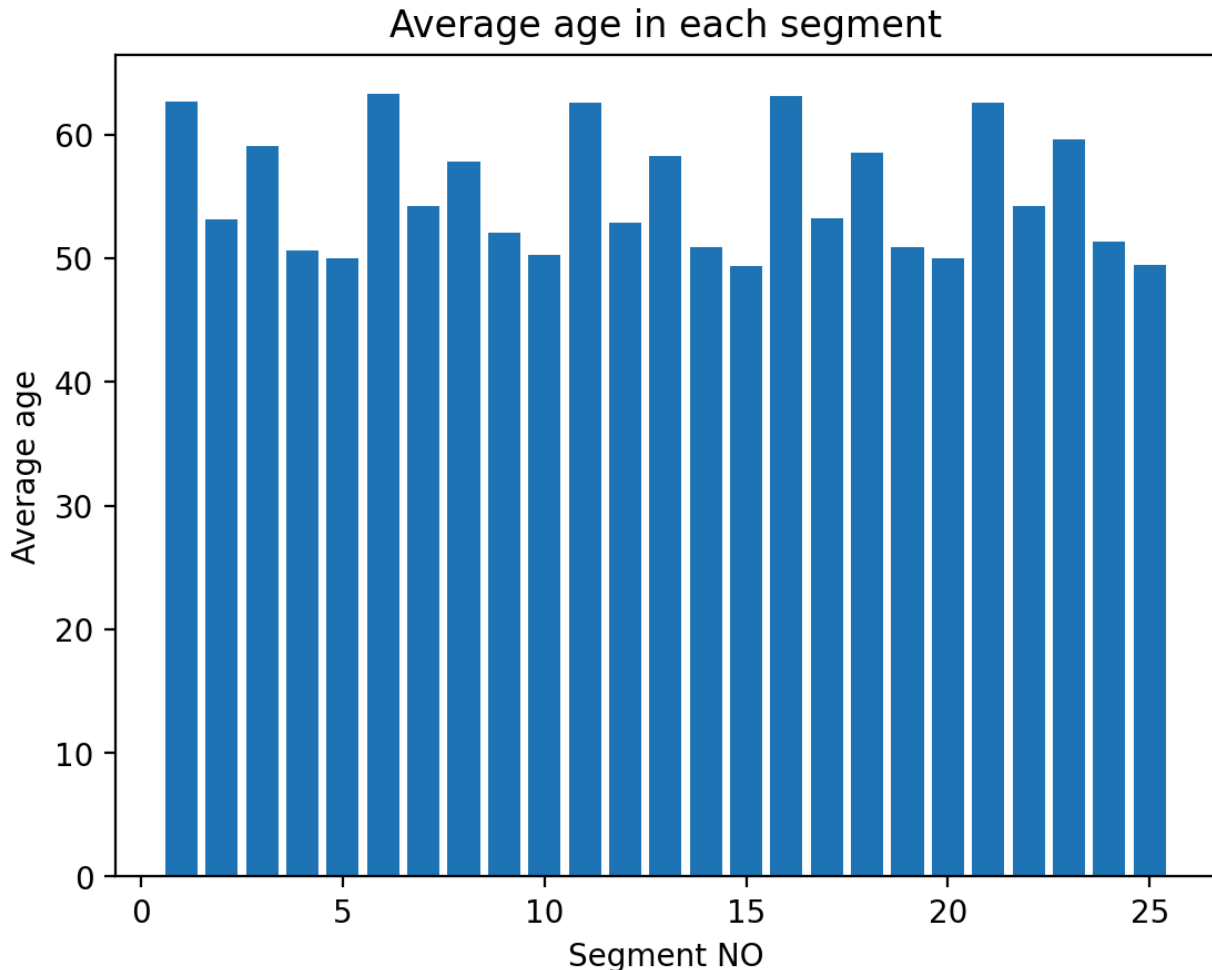
After that customers are mapped to the segments based on their risk profile and the engagement_type_id.



From the above diagram we can see that the customers are equally distributed across all the 25 segments.

AVG(AGE) in these 25 segments:

create or replace view v_age_segments as select avg(age) as average_age,segment_id from customer_segment cs join v_eligible_customers ec on ec.customer_id=cs.customer_id group by segment_id;



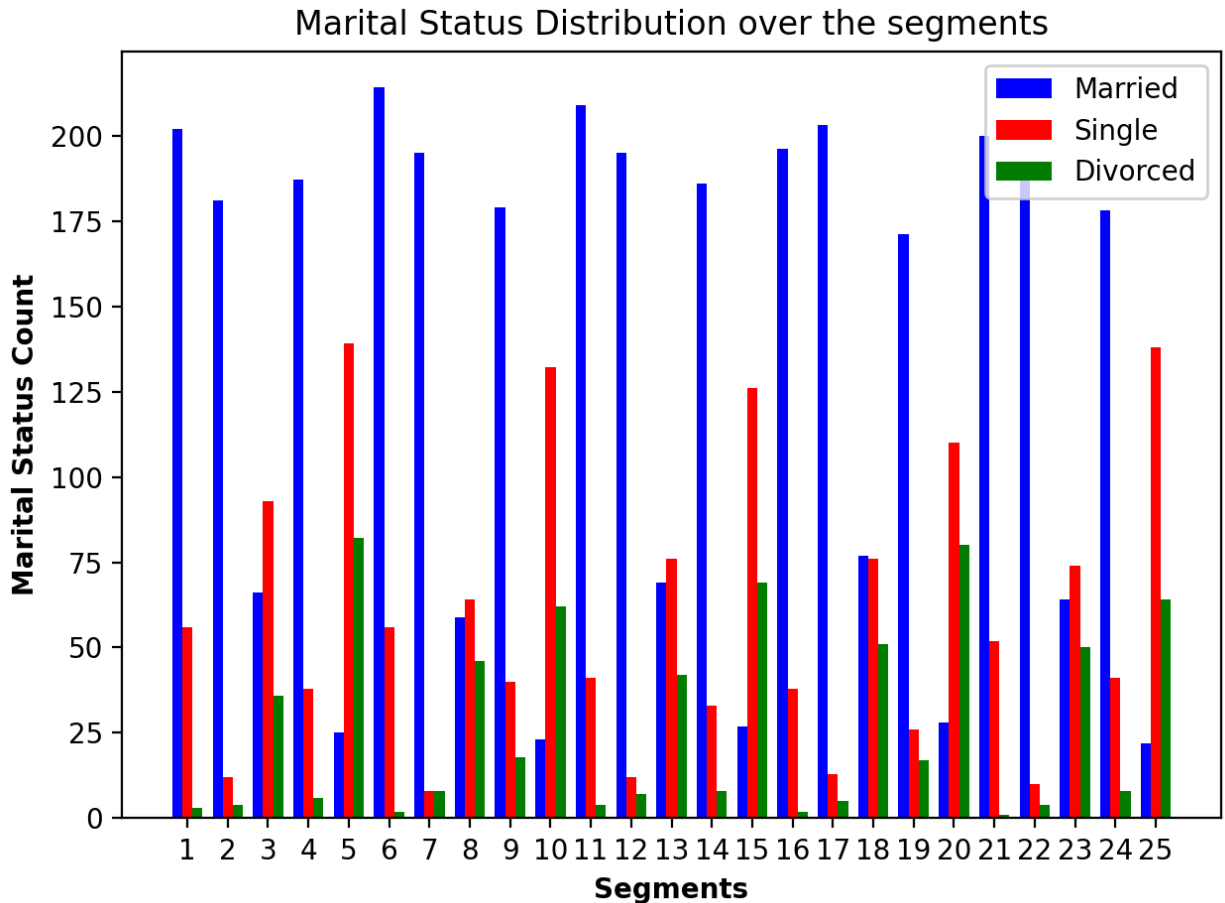
Gender(Count of MALE AND FEMALE in each segment):

create or replace view v_gender_segments as select segment_id,count(CASE WHEN gender='Male' THEN 1 END) as male_count, count(CASE WHEN gender='Female' THEN 1 END) as female_count from customer_segment cs join v_eligible_customers ec on ec.customer_id=cs.customer_id group by segment_id;

Marital Status Distribution in each Segment:

create or replace view v_marital_status_segments as select segment_id,count(CASE WHEN marital_status='Married' THEN 1 END) as

married_count, count(CASE WHEN marital_status='Single' THEN 1 END) as single_count, count(CASE WHEN marital_status='Divorced' THEN 1 END) as divorced_count from customer_segment cs join v_eligible_customers ec on ec.customer_id=cs.customer_id group by segment_id;



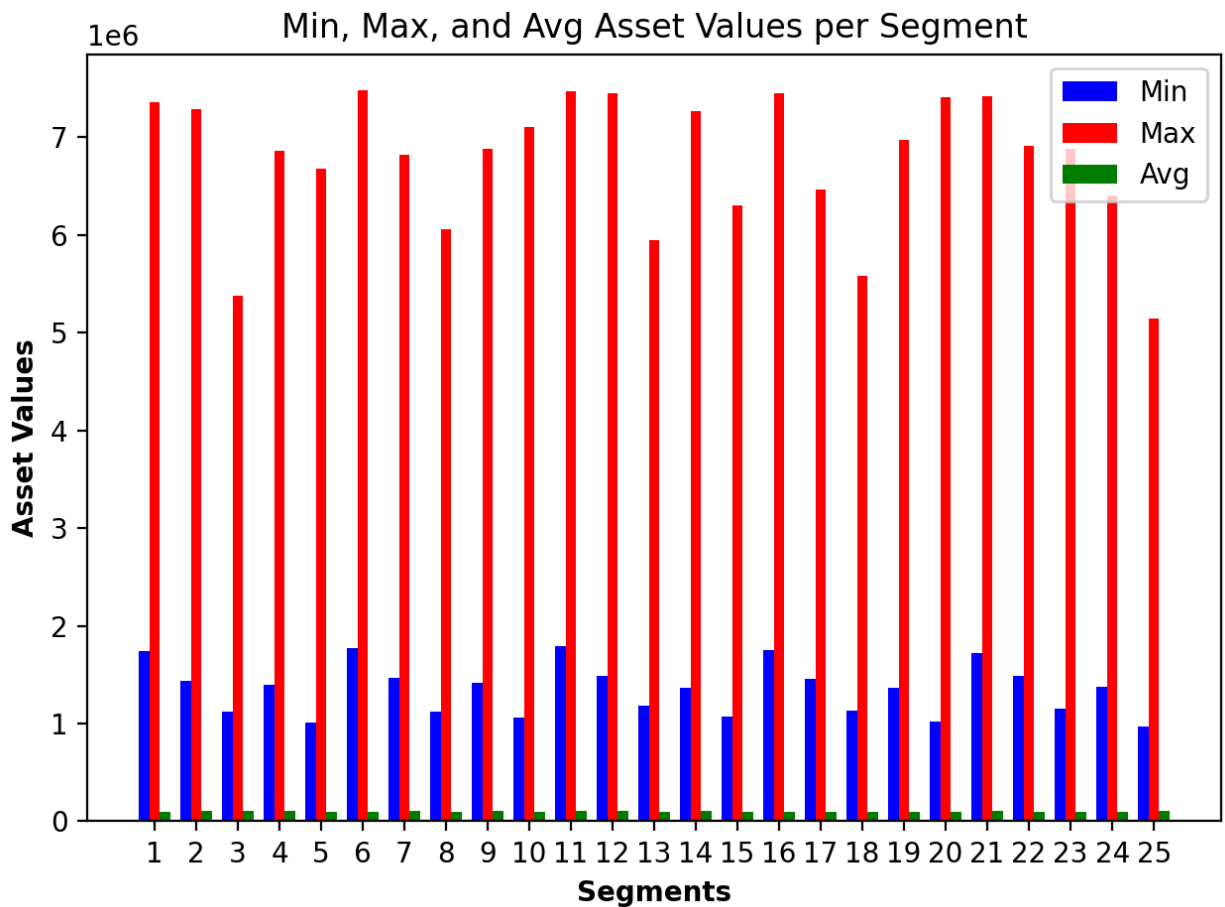
What is the range of assets available across various segments? What are the average available assets? What are the customer's maximum investable assets available in each segment?

Code:

```
create or replace view v_assets_segments as select segment_id, avg(total) as
average_assets, round(max(total), 2) as max_assets, round(min(total), 2) as
min_assets, concat(round(max(total), 2), ' - ', round(min(total), 2)) as assets_range
from customer_segment cs join v_eligible_customers ec on
```

```
ec.customer_id=cs.customer_id join customer_assets ca on
ca.customer_id=ec.customer_id group by segment_id order by segment_id asc;
```

Average assets in each segment



Scenario 2: Viewing customer segments based on risk tolerance category

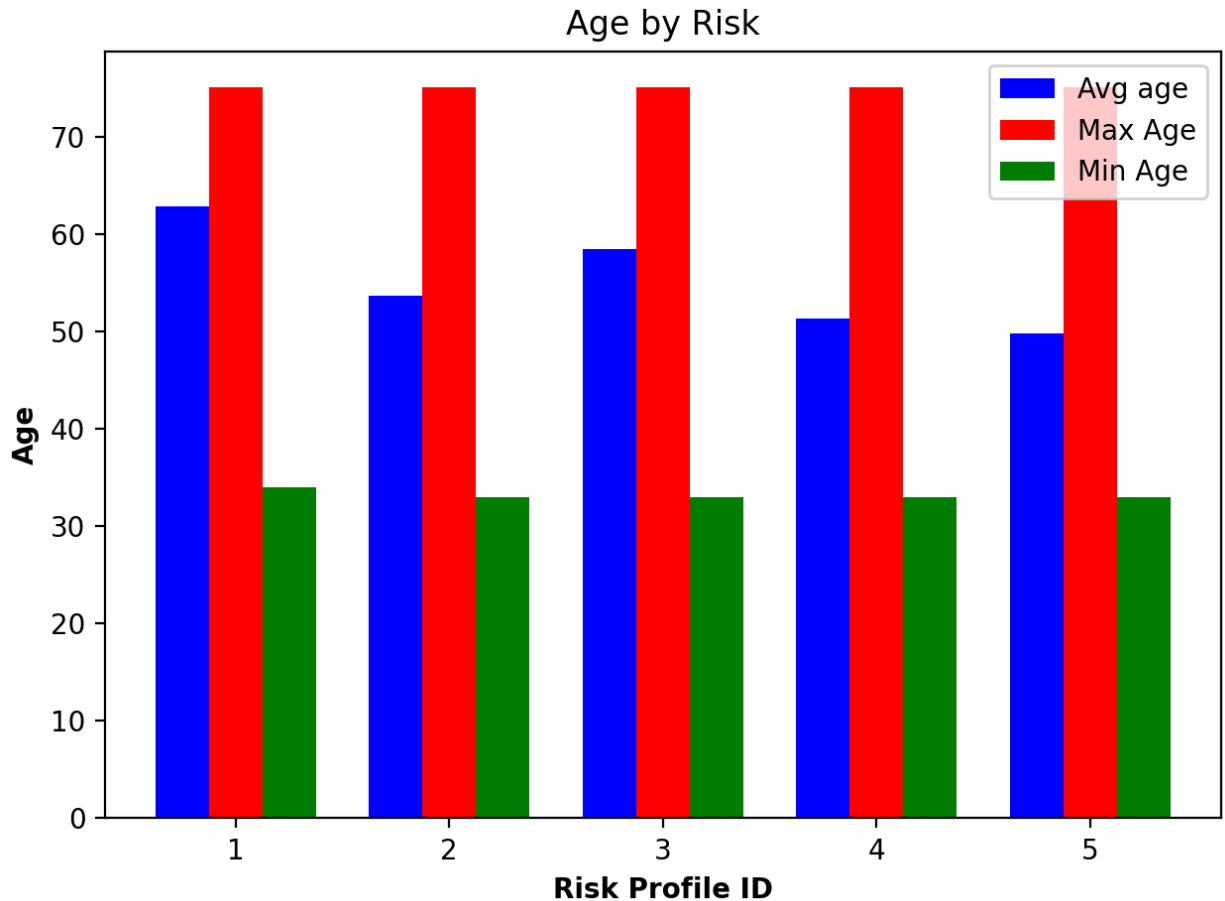
What is the relationship between the risk category and age?

create or replace view

```
CASESTUDY_WORKING.PUBLIC.V_AGE_BY_RISK(AVG_AGE,MAX_AGE
,LEAST_AGE,PCT_CST,HEAD_COUNT,RISK_PROFILE_ID) as (
```

```
select avg(age) as avg_age,max(age) as max_age, min(age) as
least_age,(count(c.customer_id) / (select count(*) from v_eligible_customers)) *
```

100 as pct_cst, count(c.customer_id) as head_count , risk_profile_id as risk_profile_id from v_eligible_customers c join v_customer_risk_profile cr on c.customer_id = cr.customer_id group by risk_profile_id order by count(c.customer_id) desc);

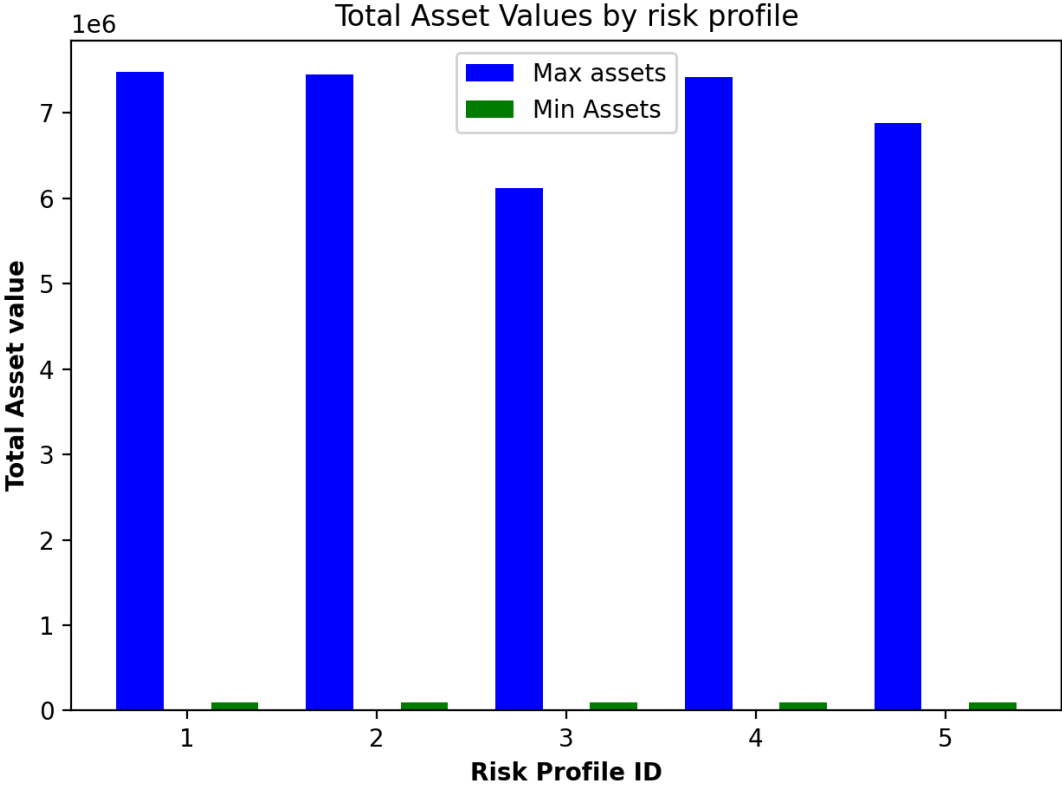


What is the relationship between available assets and risk category?

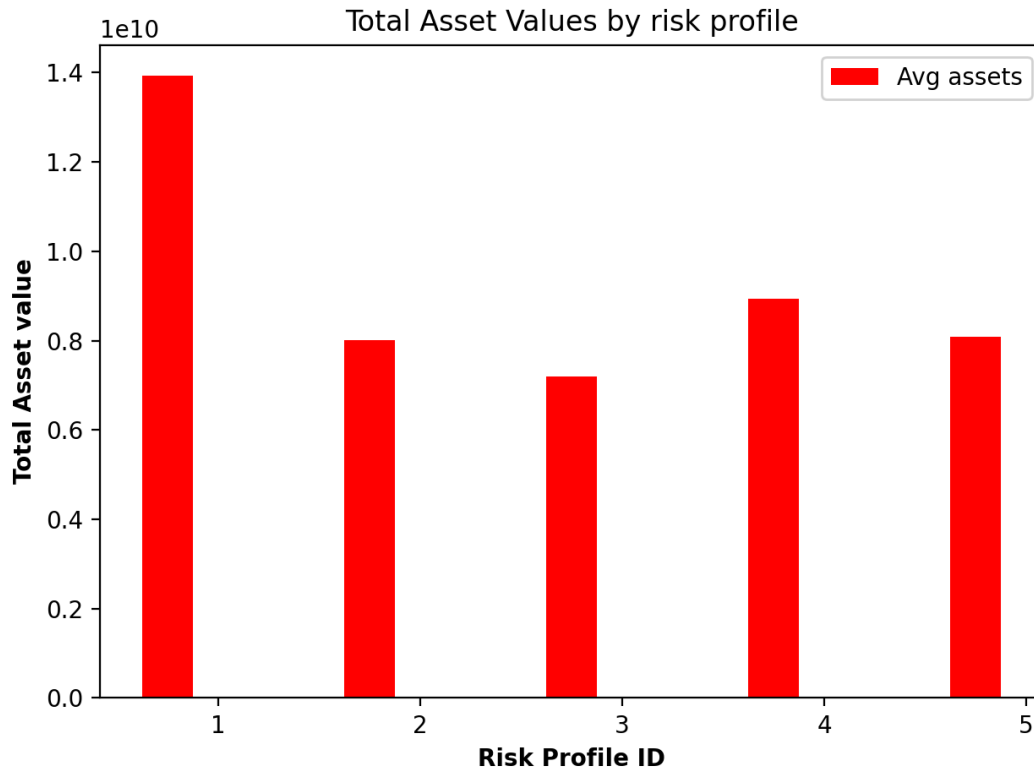
create or replace view

CASESTUDY_WORKING.PUBLIC.V_INVST_BY_RISK(AVG_INVST, MAX_INVST, MIN_INVST, PCT_CST, HEAD_COUNT, RISK_PROFILE_ID) as (select sum(total) as avg_invst, max(total) as max_invst, min(total) as min_invst, (count(c.customer_id) / (select count(*) from v_eligible_customers)) * 20 as pct_cst, count(distinct c.customer_id) as head_count , risk_profile_id as risk_profile_id from v_eligible_customers c join v_customer_risk_profile cr on c.customer_id = cr.customer_id join customer_assets ca on cr.customer_id =

ca.customer_id group by risk_profile_id order by count(c.customer_id) desc)

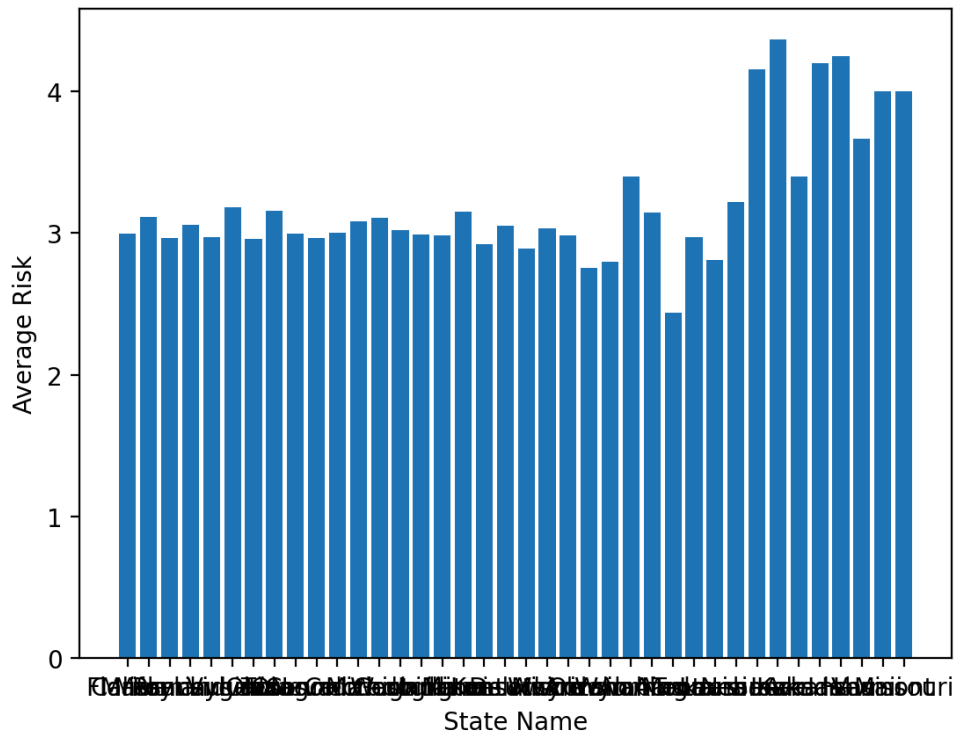


;



Are there regional variations in the distribution of risk category?

```
create or replace view v_risk_by_region as select avg(risk_profile_id) as
avg_risk,max(risk_profile_id) as max_risk, min(risk_profile_id) as
least_risk,(count(c.customer_id) / (select count(*) from v_eligible_customers)) *
100 as pct_cst, count(c.customer_id) as head_count , state as state from
v_eligible_customers c join v_customer_risk_profile cr on c.customer_id =
cr.customer_id group by state order by count(c.customer_id) desc;
```



We can see that the **average risk by region is around 2.8-3** in almost every states.

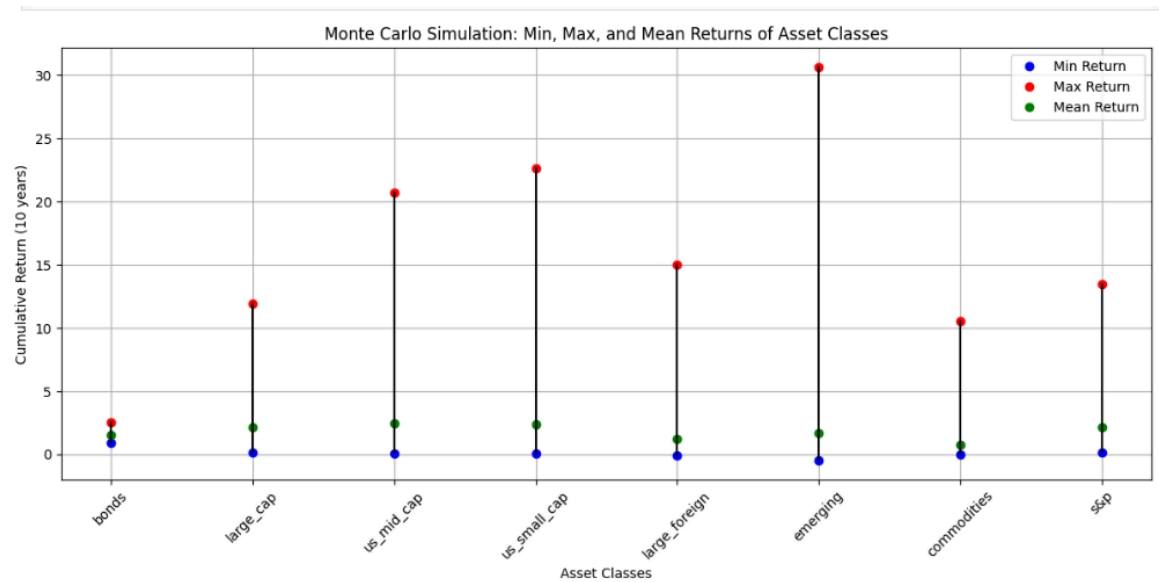
Scenario 3: Determining potential revenue for the mix of asset classes for the created funds

o Once a mix of asset classes (the fund) is chosen, what is the risk category it would be assigned? (The percentage of the fund attributed each asset class multiplied by its risk known risk category number. Round the output to a whole number. Call this the “weighted average risk score” See Appendix C.)

o For created funds calculate the approximate minimum and maximum revenue per customer (hint: multiply the fee structure by minimum investment required and maximum allowed by an individual)

o What is the potential revenue by customer segment? (hint: multiply number of customers in segment by minimum and maximum revenue)

Asset Classes Analysis by Monte Carlo Simulation



Mean. Max, Min Returns of each asset class after 1000,000 iterations

Asset Class	Min Return	Max Return	Mean Return
bonds	0.92	2.51	1.54
large_cap	0.17	11.93	2.13
us_mid_cap	0.10	20.69	2.49
us_small_cap	0.05	22.62	2.42
large_foreign	-0.06	15.03	1.25
emerging	-0.43	30.61	1.72
commodities	0.01	10.56	0.77
s&p	0.16	13.50	2.14

Fund Composition

usr/bin/python3 /Users/hardikchoudhary/Documents/Fidelity/code/fund_create.py

hardikchoudhary@Hardiks-MacBook-Air Documents % /usr/bin/python3 /Users/hardikchoudhary/Documents/Fidelity

Fund Name	Asset Allocation	Weighted Risk	Weighted Return	Weighted Fees
Risk1_Returns	bonds: 100.00%	1	4.44	1.0%
Risk1_Fees	bonds: 100.00%	1	4.44	0.9999999999999999%
Risk1_Goal	bonds: 100.00%	1	4.44	1.0%
Risk2_Returns	s&p: 30.43%	1.30434	5.48998	0.8478283725450289%
	bonds: 69.57%			
Risk2_Fees	s&p: 73.20%	1.73198	6.96533	0.634010105054004%
	bonds: 26.80%			
Risk2_Goal	bonds: 44.13%	1.55873	6.3676	0.7206369808257853%
	s&p: 55.87%			
Risk3_Returns	us_mid_cap: 64.13%	2.53849	8.29885	0.44901440704016166%
	s&p: 8.34%			
	large_cap: 8.63%			
	bonds: 18.90%			
Risk3_Fees	large_cap: 31.04%	2.86547	8.70849	0.34206618950647855%
	us_mid_cap: 60.56%			
	s&p: 3.35%			
	bonds: 5.05%			
Risk3_Goal	bonds: 5.05%	2.27957	8.26362	0.45924686420311855%
	s&p: 61.94%			
	us_mid_cap: 33.01%			
Risk4_Returns	us_mid_cap: 8.00%	2.88921	8.0412	0.46497912630616456%
	us_small_cap: 40.53%			
	s&p: 38.57%			
	large_cap: 0.35%			
	bonds: 8.53%			
	large_foreign: 4.02%			
Risk4_Fees	large_cap: 10.95%	3.11849	5.96349	0.6583588771667784%
	us_mid_cap: 24.15%			
	us_small_cap: 12.93%			
	s&p: 1.08%			
	bonds: 16.96%			
	large_foreign: 33.92%			
Risk4_Goal	bonds: 64.61%	1.64244	6.13991	0.7653455220984878%
	s&p: 6.53%			
	us_mid_cap: 28.86%			
Risk5_Returns	us_mid_cap: 13.08%	3.29508	7.41437	0.5308980775952754%
	us_small_cap: 40.02%			
	s&p: 16.28%			
	large_cap: 4.25%			
	emerging: 2.76%			
	bonds: 7.78%			
	large_foreign: 15.82%			
Risk5_Fees	large_cap: 14.48%	2.69348	7.00805	0.5592628375575042%
	us_mid_cap: 7.89%			
	us_small_cap: 18.25%			
	s&p: 31.27%			
	bonds: 15.24%			
	large_foreign: 12.86%			
Risk5_Goal	bonds: 23.67%	2.66608	7.17242	0.5268814238103404%
	s&p: 8.06%			
	us_mid_cap: 16.06%			
	large_cap: 30.21%			
	us_small_cap: 15.57%			
	large_foreign: 6.44%			

hardikchoudhary@Hardiks-MacBook-Air Documents %

Funds Analysis

Fund creation by maximizing returns in each category and keeping the condition (each asset should be greater than 1% and a fund should have at least 4 asset classes out of 8)

Fund 1 (Target Risk Profile: 1):
Actual Risk Profile: 1.19
Expected Return: 4.52%
Asset Allocation:
bonds: 93.00%
large_cap: 1.00%
us_mid_cap: 1.00%
us_small_cap: 1.00%
large_foreign: 1.00%
emerging: 1.00%
commodities: 1.00%
s&p: 1.00%

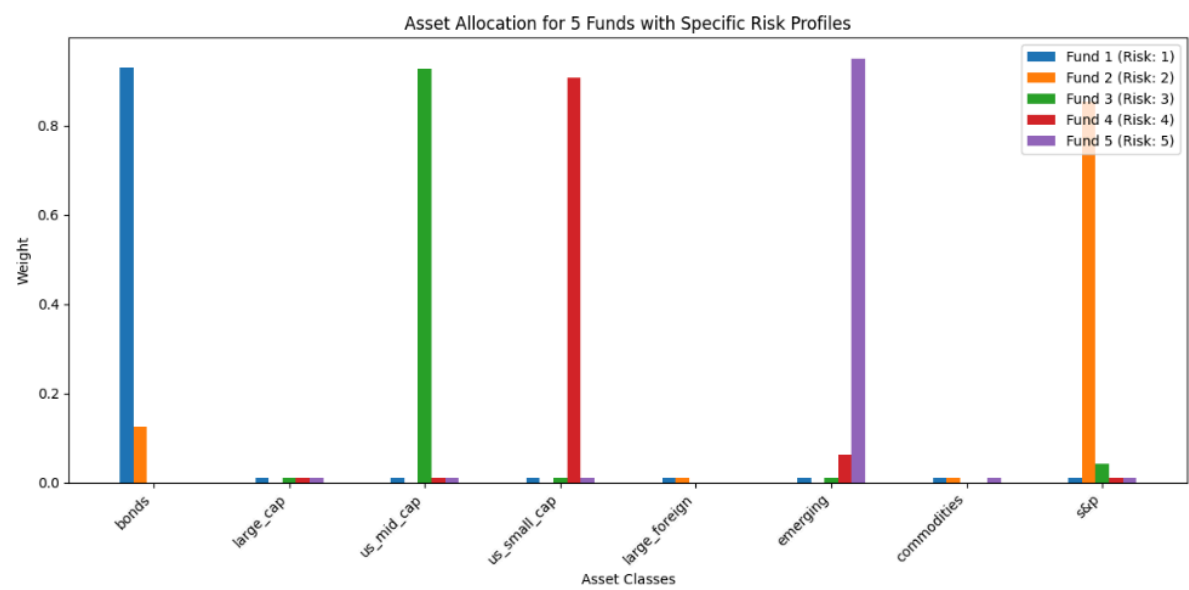
Fund 2 (Target Risk Profile: 2):
Actual Risk Profile: 1.93
Expected Return: 7.33%
Asset Allocation:
bonds: 12.50%
large_foreign: 1.04%
commodities: 1.04%
s&p: 85.42%

Fund 3 (Target Risk Profile: 3):
Actual Risk Profile: 2.99
Expected Return: 9.18%
Asset Allocation:
large_cap: 1.03%
us_mid_cap: 92.78%
us_small_cap: 1.03%
emerging: 1.03%
s&p: 4.12%

Fund 4 (Target Risk Profile: 4):
Actual Risk Profile: 4.02
Expected Return: 8.84%
Asset Allocation:
large_cap: 1.03%
us_mid_cap: 1.03%
us_small_cap: 90.72%
emerging: 6.19%
s&p: 1.03%

Fund 5 (Target Risk Profile: 5):
Actual Risk Profile: 4.92
Expected Return: 5.56%
Asset Allocation:
large_cap: 1.02%
us_mid_cap: 1.02%
us_small_cap: 1.02%
emerging: 94.90%
commodities: 1.02%
s&p: 1.02%

Assets in each funds



1. Fund Creation based on Fee Optimisation

Fund 1 (Risk Profile: 1, Max Fee: 0.50%):

Actual Risk Profile: 1.69

Expected Return: 6.34%

Actual Fee: 0.77%

Asset Allocation:

bonds: 41.39%

large_cap: 1.01%

us_mid_cap: 1.01%

us_small_cap: 1.01%

emerging: 1.01%

commodities: 1.01%

s&p: 53.56%

Fund 2 (Risk Profile: 2, Max Fee: 0.75%):

Actual Risk Profile: 1.95

Expected Return: 7.44%

Actual Fee: 0.57%

Asset Allocation:

bonds: 12.24%

large_cap: 1.02%

us_mid_cap: 1.02%

us_small_cap: 1.02%

emerging: 1.02%

s&p: 83.67%

Fund 3 (Risk Profile: 3, Max Fee: 1.00%):

Actual Risk Profile: 2.99

Expected Return: 9.35%

Actual Fee: 0.32%

Asset Allocation:

large_cap: 1.03%

us_mid_cap: 92.78%

us_small_cap: 1.03%

emerging: 1.03%

s&p: 4.12%

Fund 4 (Risk Profile: 4, Max Fee: 1.25%):

Actual Risk Profile: 4.02

Expected Return: 9.02%

Actual Fee: 0.38%

Asset Allocation:

large_cap: 1.03%

us_mid_cap: 1.03%

us_small_cap: 90.72%

emerging: 6.19%

s&p: 1.03%

Fund 5 (Risk Profile: 5, Max Fee: 1.50%):

Actual Risk Profile: 4.95

Expected Return: 5.39%

Actual Fee: 1.54%

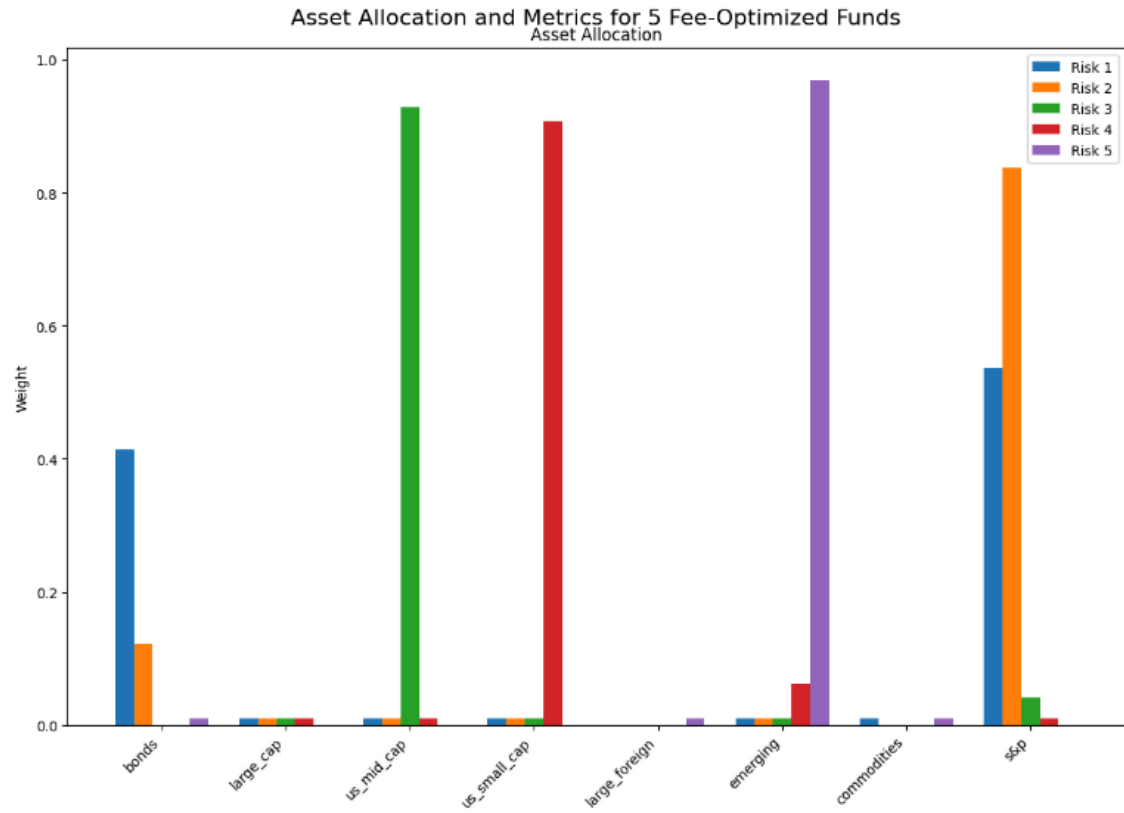
Asset Allocation:

bonds: 1.04%

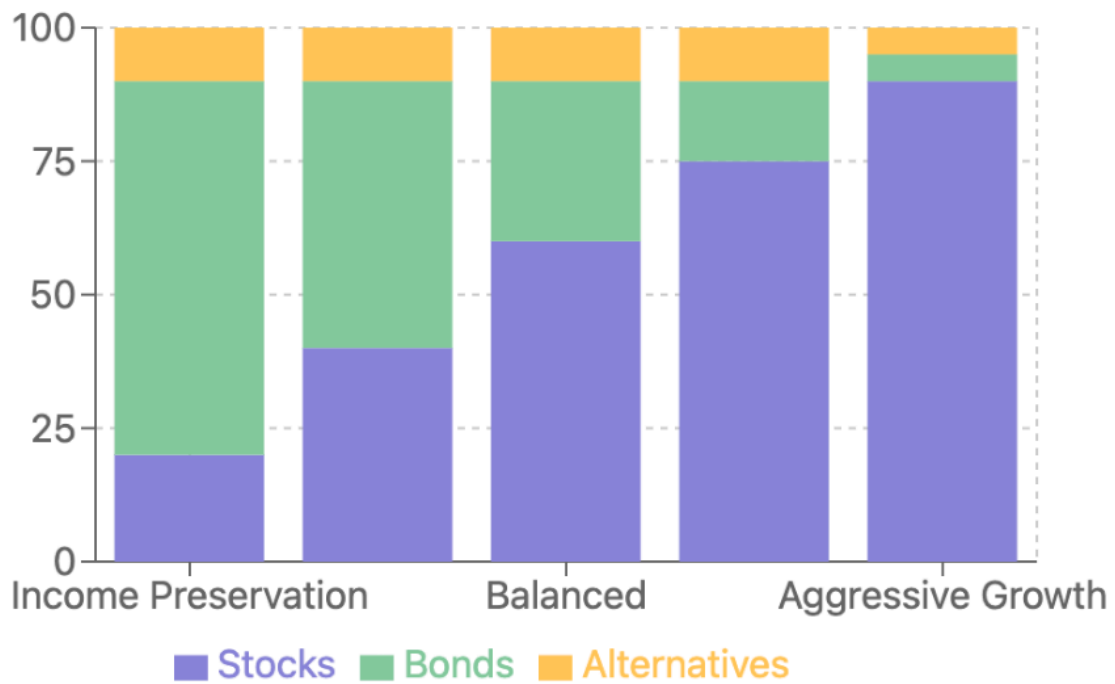
large_foreign: 1.04%

emerging: 96.87%

commodities: 1.04%



2. Fund based on asset objective



Retirement Builder:
Risk Score: 3.00
Expected Return: 8.40%
Fee: 0.44%
Asset Allocation:
bonds: 20.00%
large_cap: 0.00%
us_mid_cap: 40.00%
us_small_cap: 40.00%
large_foreign: 0.00%
emerging: 0.00%
commodities: 0.00%
s&p: 0.00%

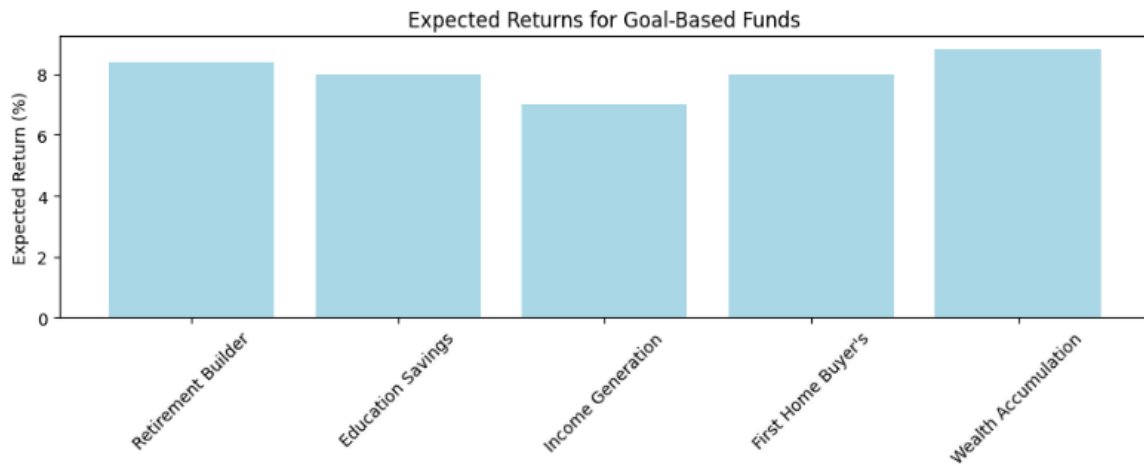
Education Savings:
Risk Score: 2.50
Expected Return: 7.98%
Fee: 0.51%
Asset Allocation:
bonds: 30.00%
large_cap: 0.00%
us_mid_cap: 60.00%
us_small_cap: 10.00%
large_foreign: 0.00%
emerging: 0.00%
commodities: 0.00%
s&p: 0.00%

Income Generation:
Risk Score: 2.00
Expected Return: 7.00%
Fee: 0.65%
Asset Allocation:
bonds: 50.00%
large_cap: 0.00%
us_mid_cap: 50.00%
us_small_cap: 0.00%
large_foreign: 0.00%
emerging: 0.00%
commodities: 0.00%
s&p: 0.00%

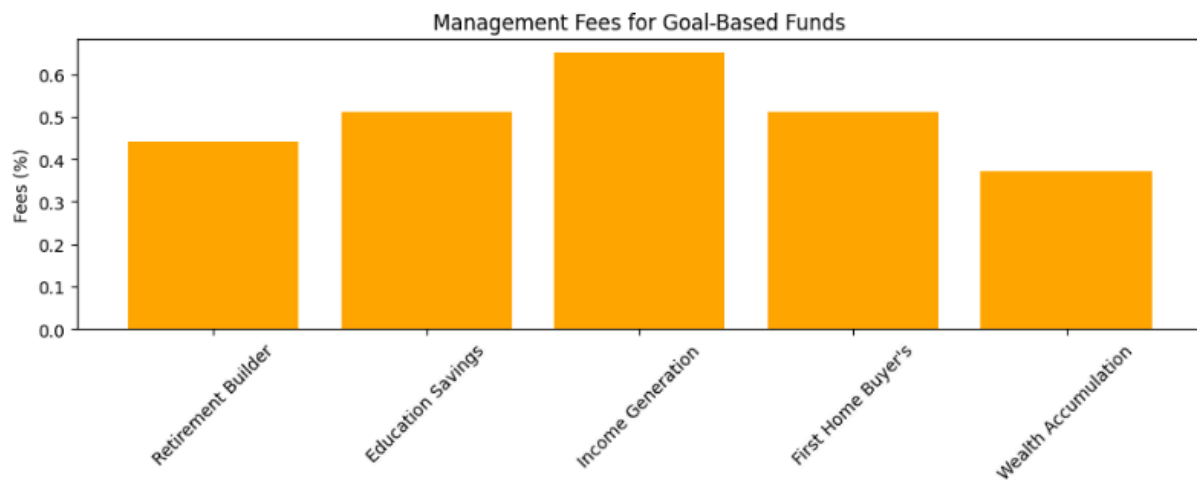
First Home Buyer's:
Risk Score: 2.50
Expected Return: 7.98%
Fee: 0.51%
Asset Allocation:
bonds: 30.00%
large_cap: 0.00%
us_mid_cap: 60.00%
us_small_cap: 10.00%
large_foreign: 0.00%
emerging: 0.00%
commodities: 0.00%
s&p: 0.00%

Wealth Accumulation:
Risk Score: 3.50
Expected Return: 8.81%
Fee: 0.37%
Asset Allocation:
bonds: 10.00%
large_cap: 0.00%
us_mid_cap: 20.00%
us_small_cap: 70.00%

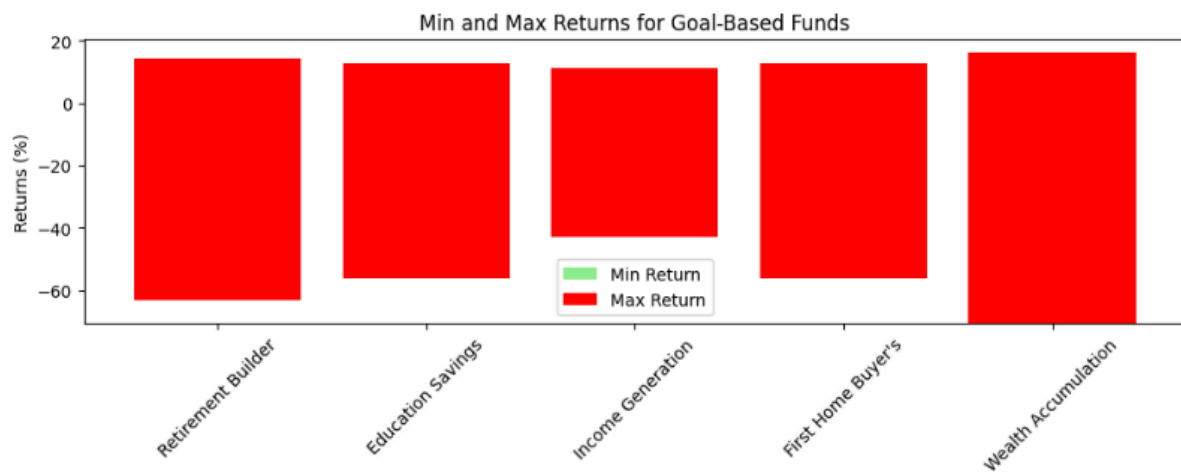
Mean Returns of funds based on asset objective



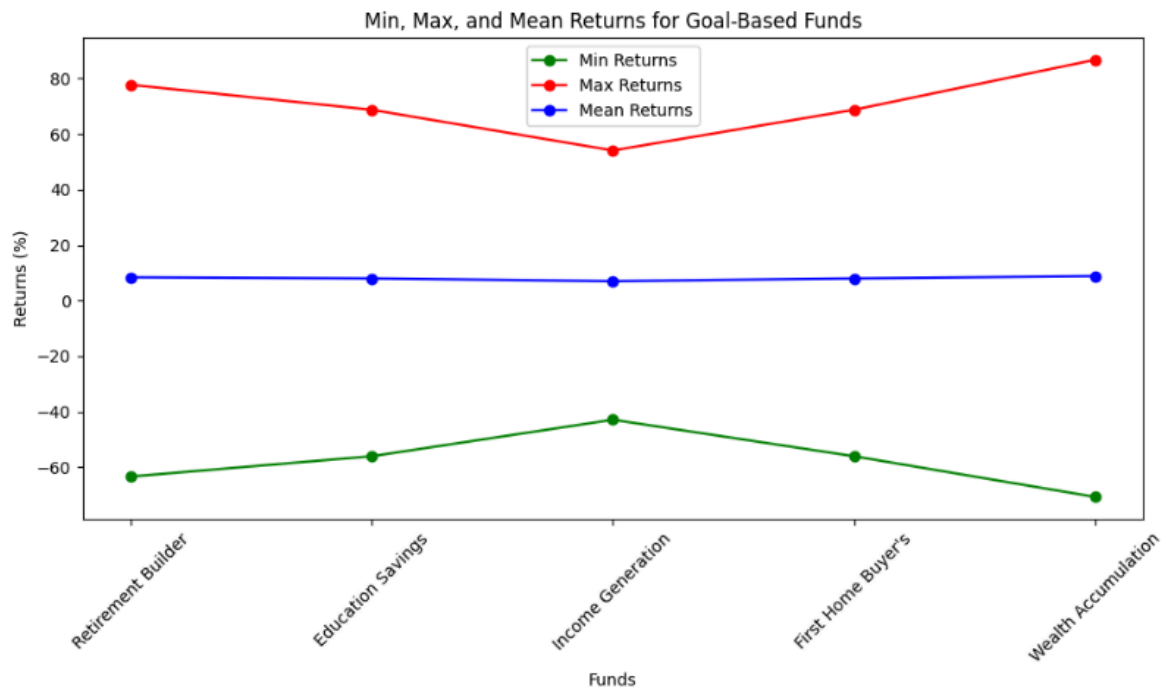
Fees on funds based on asset objective



Max Drawdowns for the funds



Min, Max, Mean returns of funds based on asset objective



3. Fees and Returns Corelation

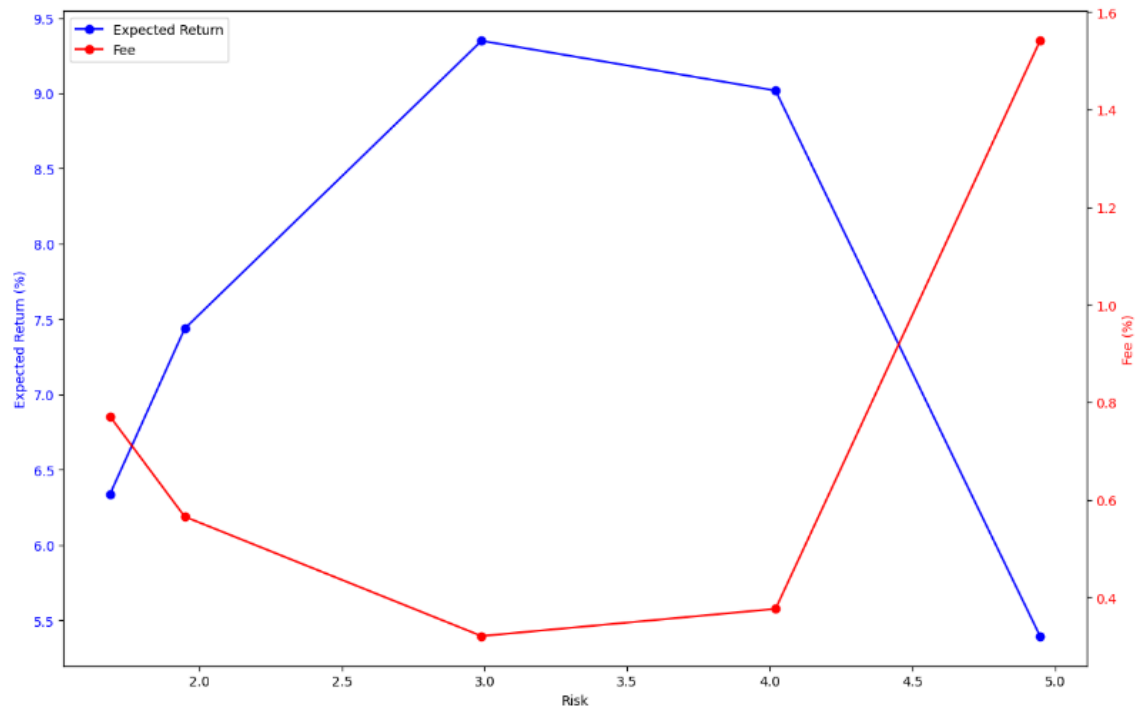
Overall Fee Analysis:

Min Fee: 0.37%

Max Fee: 0.65%

Mean Fee: 0.50%

Comparative analysis of fees and returns



Key Takeaway:

- We are able to find two point of intersection which are in conservative (1.5) and aggressive risk profile(4.5). We will try to these profiles from our analysis till now as they are beneficial for customers as well as the firm.
- This graph provides a good picture of average fees (revenue) the firm will be making.

Produce a series of functions and procedures to support dynamic reporting in later activities

1. Function get_age_gnder_segments:

create or replace function get_segment(gender varchar(10),age_range varchar(20))

returns number

language sql

as

\$\$

```

case when gender='Male' and age_range='31-40' then 1
when gender='Female' and age_range='31-40' then 2
when gender='Male' and age_range='41-50' then 3
when gender='Female' and age_range='41-50' then 4
when gender='Male' and age_range='51-60' then 5
when gender='Female' and age_range='51-60' then 6
when gender='Male' and age_range='61-70' then 7
when gender='Female' and age_range='61-70' then 8
when gender='Male' and age_range='71-80' then 9
when gender='Female' and age_range='71-80' then 10
end
$$
;

```

- It returns the segment_id based on gender and age_range.

Steps

- Create functions for each of the calculations used in the views in the previous activity. At minimum include functions for the following:
- **Assigning customers to a risk tolerance category by using a rounded average of the customer's answers to the 8 questions in the risk questionnaire data (the risk questionnaire questions can be found in Appendix E)**

- Calculating the minimum and maximum potential revenue per customer for the potential fund view(s)
- Incorporate the returns listed in the table below.
- Calculating the weighted average risk score of your fund samples
- Create procedure toolsets for managing the implementation of CRUD to manipulate data within the tables you have created.
- Make sure to incorporate code to protect your tables from bad data in accordance with your profiling work.
- As time permits put in the following triggers and check constraints to manage or improve data integrity
- Ensure the percentage distribution within the fund assets table cannot exceed 100%
- Create a formatted mailing address which has first and last name on the first line, street address on line two, and City, State Zip on line three
- Create a duplicate detection and remediation routine

Phase 2

Deliverables

- Customer segmentation
- Risk Tolerance analysis

Steps

- In Excel, perform analysis of the provided csv data from a customer perspective.
- You should be looking to identify at least 5 core customer segments your team is interested in exploring as potential targets for your eventual fund.

- Look to use your analysis as a way of differentiating between these segments and identifying key advantages and disadvantages of each.
- Once you have a handle on the customer data it will be important for your team to analyze the risk data provided. Every customer took a risk tolerance survey and we need to make sure our eventual fund recommendations take these risk preferences into account.
- Again using excel perform analysis on this risk data to determine if there are any critical elements to the data you should be taking into consideration
- If your team has comfort with Python you may find it useful to create a new Jupyter notebook and import the provided csv data using python.
- Once the data is imported you can use the available python libraries and visualization tools to begin exploring the data further.
- Remember, a big part of the early goal here is for your team to become comfortable with some core elements of the data you will be working with. Begin the process of organizing, exploring, and documenting your findings for posterity.

Phase 3

Deliverables

- You will be using your Python skills to access and extrapolate information from the provided data.
- At the end of this session your team should have a well-documented data dictionary for each of the datasets provided.

Using python we have created dictionaries for all the data that has been provided for us.

- Use the information gleaned from your lessons up until this point to determine the most important aspects of your data to analyze and comment on. What are some of the fundamental aspects of our data that can affect our ongoing analysis?

Steps

- Build a jupyter notebook that will act as your ongoing data dictionary/Data Profile for your dataset

Markdown ▾ as `cell6`

Customers Table

Customer_ID = Unique Identifier for the customers(Primary Key)

Contact_Last_Name=Last Name of the Customer

Contact_First_Name= First Name of the Customer

Street = Street Name

City = The City in which the customer is residing

Zip = The Zip Code of the city

Country = The Country of the customer

Age = The age of the customer

Marital Status = This column holds whether the customer is single/married/divorced/Unknown.

Gender = This column holds whether the customer is male/female/others.

No of Dependents = This column contains the No of Persons who are depending on the customers.

Questions Table

Question ID - Unique Identifier for the question(Primary Key) Question Subject - The subject of the question(motive of the question).

Question Text - The actual question that will be asked to the customers.

Example: For the questions table, we have created a dictionary and appended all the data into it.

```

[
  0 : {
    "QUESTION_SUBJECT" : "Years Experience"
    "QUESTION_TEXT" :
      "How many years from now do you expect to begin taking income from your investments?"
    "QUESTION_ID" : 1
  }
  1 : {
    "QUESTION_SUBJECT" : "Initial Loss Behavior"
    "QUESTION_TEXT" :
      "Choose the BEST match to your feelings on the following situation: Your account value after
      rising 15% in the previous quarter is down 20% this quarter."
    "QUESTION_ID" : 2
  }
  2 : {
    "QUESTION_SUBJECT" : "Subsequent Loss Behavior"
    "QUESTION_TEXT" :
      "What if your portfolio's value declined another 10% in the subsequent quarter?"
    "QUESTION_ID" : 3
  }
  3 : {

```

- Explore the idiosyncrasies of each table in the dataset

Datatype of the columns:

```

1 dfcustomers=cell2.to_pandas()
2 print("Datatype of all the columns:",dfcustomers.dtypes)

```

```

Datatype of all the columns: CUSTOMER_ID          int16
CONTACT_LAST_NAME      object
CONTACT_FIRST_NAME     object
STREET                 object
CITY                   object
STATE                  object
ZIP                    int32
COUNTRY                object
AGE                    int8
MARITAL_STATUS         object
GENDER                 object
NUMBER_OF_DEPENDENTS   int8
dtype: object

```

No of missing values that are present in each column:

```
1 | print("Missing values count:",dfcustomers.isnull().sum())
```

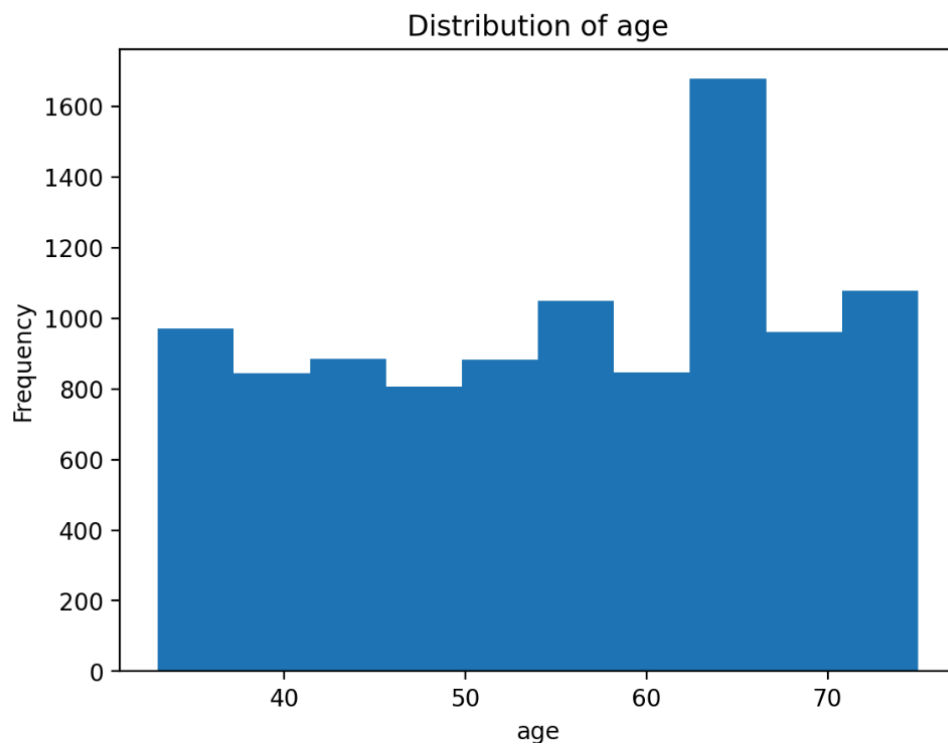
```
Missing values count: CUSTOMER_ID          0
CONTACT_LAST_NAME      0
CONTACT_FIRST_NAME     0
STREET                 0
CITY                   0
STATE                  0
ZIP                    0
COUNTRY                0
AGE                    0
MARITAL_STATUS        1889
GENDER                 990
NUMBER_OF_DEPENDENTS   0
dtype: int64
```

Here we can see that marital status and gender are the columns that consists of many no of missing values.

Data distribution and summary statistics:

```
1 | print("Statistical summary for the column age")
2 | print(dfcustomers['AGE'].describe())
```

```
Statistical summary for the column age
count    10000.000000
mean      55.295100
std       12.140517
min       33.000000
25%       45.000000
50%       57.000000
75%       65.000000
max       75.000000
Name: AGE, dtype: float64
```

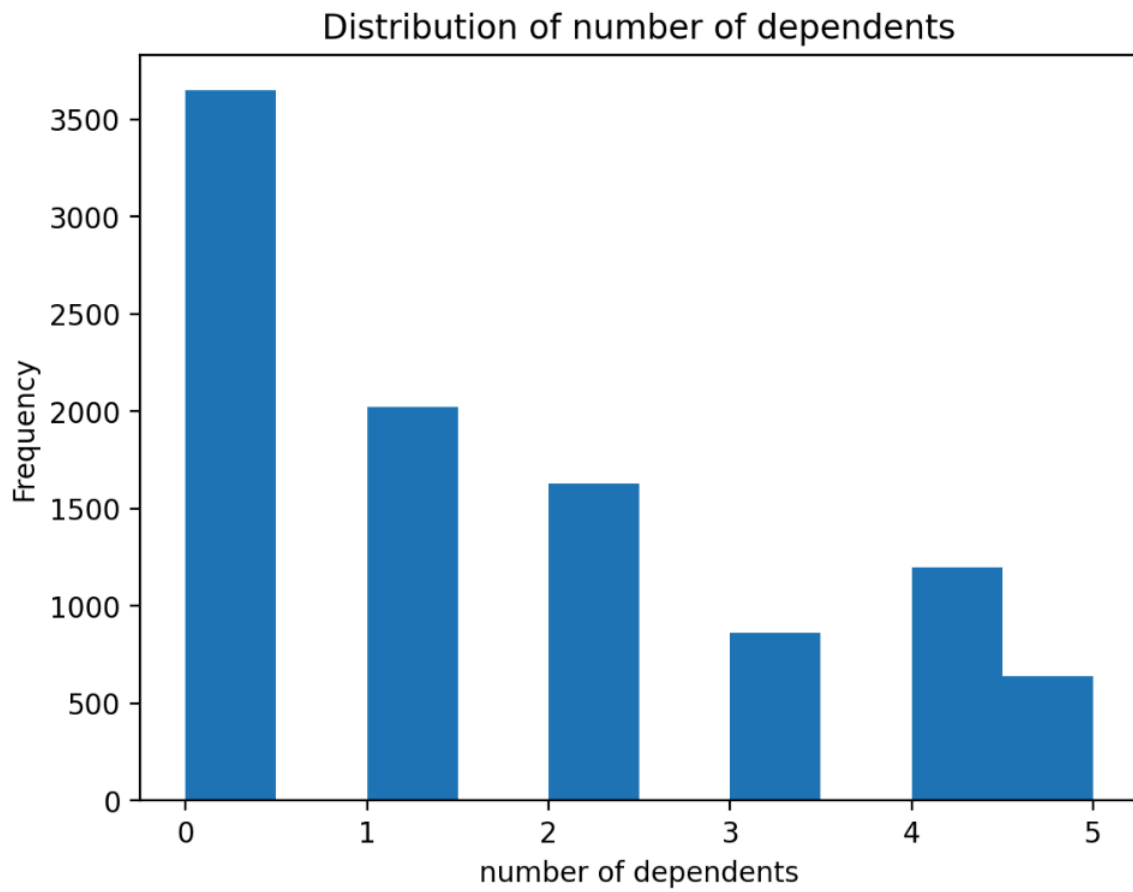


```
1 print("Statistical summary for the column age")
2 print(dfcustomers['NUMBER_OF_DEPENDENTS'].describe())
```

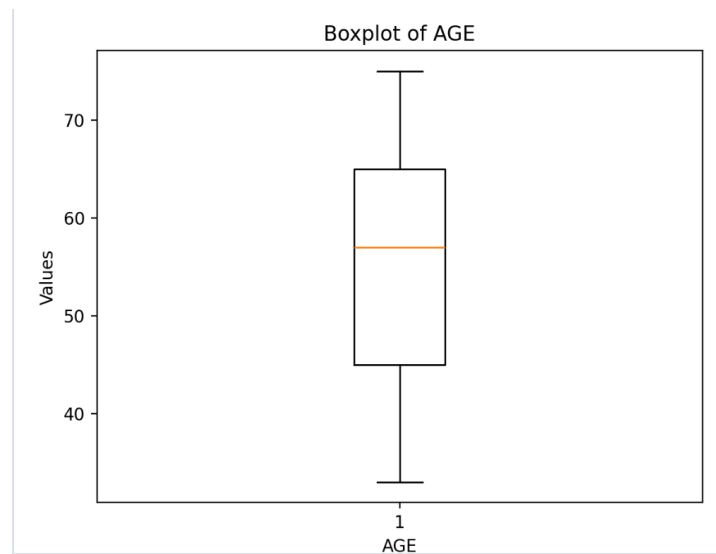
Statistical summary for the column age

count	10000.000000
mean	1.586200
std	1.622662
min	0.000000
25%	0.000000
50%	1.000000
75%	3.000000
max	5.000000

Name: NUMBER_OF_DEPENDENTS, dtype: float64



· **What data anomalies exist?**



From the box plot, we can conclude that there are no outliers present in the age column. To confirm this, the IQR range is found and the outliers are further checked.

Python ▾ as cell57

0.0s ▶ 🔍

```
1 def find_anomalies_iqr(column):
2     Q1 = column.quantile(0.25)
3     Q3 = column.quantile(0.75)
4     IQR = Q3 - Q1
5     lower_bound = Q1 - 1.5 * IQR
6     upper_bound = Q3 + 1.5 * IQR
7     outliers = column[(column < lower_bound) | (column > upper_bound)]
8     return outliers
```

Python ▾ as cell59

```
1 outliers = find_anomalies_iqr(dfcustomers['AGE'])
2 print("Outliers in age:\n", outliers)
```

Outliers in age:

Series([], Name: AGE, dtype: int8)

Distribution of the categorical data:

```

1 print("Distribution of no of dependents:",dfcustomers['NUMBER_OF_DEPENDENTS'].value_counts())
2 print("Distribution of marital status:",dfcustomers['MARITAL_STATUS'].value_counts())
3 print("Distribution of gender:",dfcustomers['GENDER'].value_counts())

```

Distribution of no of dependents: NUMBER_OF_DEPENDENTS

0	3648
1	2023
2	1629
4	1199
3	860
5	641

Name: count, dtype: int64

Distribution of marital status: MARITAL_STATUS

Married	4370
Single	2312
Divorced	1429

Name: count, dtype: int64

Distribution of gender: GENDER

Male	5909
Female	3101

Name: count, dtype: int64

- cell17
- cell18
- cell19
- cell20
- cell21
- cell22
- cell23
- cell24
- cell25
- cell26
- cell27
- cell28
- cell29
- cell30
- cell31
- cell32
- cell33
- cell34
- cell35
- cell36
- cell37

Phase 4

Analyze the provided news dataset to understand its nature and key features.

Importing the necessary libraries:

Spacy: SpaCy is an open-source Python natural language processing (NLP) module that is intended to parse text quickly and effectively. It offers functions including text pre-processing, dependency parsing, named entity recognition, and part-of-speech tagging. Building NLP apps and models is a common usage for SpaCy because of its flexible API and multilingual capabilities.

```
: pip install scikit-learn
```

```
Requirement already satisfied: scikit-learn in c:\users\administrator\appdata\local\program
es (1.5.2)
Requirement already satisfied: joblib>=1.2.0 in c:\users\administrator\appdata\local\progra
ges (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\administrator\appdata\local
e-packages (from scikit-learn) (3.5.0)
Requirement already satisfied: scipy>=1.6.0 in c:\users\administrator\appdata\local\program
es (from scikit-learn) (1.14.1)
Requirement already satisfied: numpy<1.10.0, >=1.9.0 in c:\users\administrator\appdata\local\progra
```

```
: pip install spacy
```

```
Collecting spacy
  Using cached spacy-3.7.6-cp310-cp310-win_amd64.whl (12.1 MB)
Collecting pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4
  Using cached pydantic-2.9.2-py3-none-any.whl (434 kB)
Collecting preshed<3.1.0,>=3.0.2
  Using cached preshed-3.0.9-cp310-cp310-win_amd64.whl (122 kB)
Requirement already satisfied: setuptools in c:\users\administrator\appdata\local\programs\python\python310\lib\si
(from spacy) (63.2.0)
Collecting srsly<3.0.0,>=2.4.3
  Using cached srsly-2.4.8-cp310-cp310-win_amd64.whl (481 kB)
Collecting langcodes<4.0.0,>=3.2.0
  Using cached langcodes-3.4.0-py3-none-any.whl (182 kB)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in c:\users\administrator\appdata\local\programs\python
\lib\site-packages (from spacy) (3.0.12)
Requirement already satisfied: numpy>=1.19.0 in c:\users\administrator\appdata\local\programs\python\python310\lib
ges (from spacy) (1.26.4)
Collecting thinc<8.3.0,>=8.2.2
  Using cached thinc-8.2.5-cp310-cp310-win_amd64.whl (1.5 MB)
Collecting murmurhash<1.1.0,>=0.28.0
  Using cached murmurhash-1.0.10-cp310-cp310-win_amd64.whl (25 kB)
Requirement already satisfied: iinia2 in c:\users\administrator\appdata\local\programs\python\python310\lib\site-pa
```

```
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
n [10]: !python -m spacy download en_core_web_lg
```

```
Collecting en-core-web-lg==3.7.1
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_lg-3
any.whl (587.7 MB)
----- 587.7/587.7 MB 1.3 MB/s eta 0:00:00
Requirement already satisfied: spacy<3.8.0,>=3.7.2 in c:\users\administrator\appdata\local
-packages (from en-core-web-lg==3.7.1) (3.7.6)
Requirement already satisfied: Jinja2 in c:\users\administrator\appdata\local\programs\pyt
om spacy<3.8.0,>=3.7.2->en-core-web-lg==3.7.1) (3.1.4)
```

SpaCy offers a **pre-trained English language model** called **en_core_web_lg** that is intended for a variety of natural language processing applications. It can perform better on tasks like named entity recognition, part-of-speech tagging, and similarity

comparisons because of its rich lexicon and dense word vectors. This model is appropriate for applications needing in-depth comprehension of English text since it is accurate and capable of handling complicated linguistic aspects. Because of its effective design, it also provides higher processing rates, meeting the demands of both manufacturing and research.

```
col = ['Date', 'News']  
data = pd.read_csv(r"C:\Users\Administrator\751809\Fid-Case-Study\sentiment_analysis\RedditNews.csv", names
```

```
data.head()
```

	Date	News
0	Date	News
1	2016-07-01	A 117-year-old woman in Mexico City finally re...
2	2016-07-01	IMF chief backs Athens as permanent Olympic host
3	2016-07-01	The president of France says if Brexit won, so...
4	2016-07-01	British Man Who Must Give Police 24 Hours' Not...

```
data.shape
```

```
(73609, 2)
```

```
data.info()
```

After downloading the necessary libraries and the language model, the exploration of the news data is done. The news data contains two columns namely the date-the date of the news and the second column contains the news in that date. There are 73609 rows in the csv file.

```
- -  
:  
data.dropna(inplace=True)
```

Dropping the missing values from the news data.

```

nlp = spacy.load("en_core_web_lg")
def preprocess(text):
    doc = nlp(text)
    filtered_tokens = []
    for token in doc:
        if token.is_stop or token.is_punct:
            continue
        filtered_tokens.append(token.lemma_)
    #print("preprocess successful")
    return " ".join(filtered_tokens)

```

The above function processes the input text to create a list of filtered tokens, removing stop words and punctuation. The function lemmatizes the remaining tokens (reducing words to their base forms) and returns them as a single space-separated string. This preprocessing step is essential for preparing text data for further analysis or natural language processing tasks.

```

start_time = time.time()
print(start_time)
data['Preprocessed Text'] = data['News'].apply(preprocess)
end_time = time.time()
total_time_taken = end_time - start_time
print(total_time_taken)

```

Using the preprocess function that is defined in the prior step, all the news data is preprocessed and the total time taken for preprocessing the data is calculated.

```

try:
    data.to_csv(r"C:\Users\Administrator\751809\Fid-Case-Study\sentiment_analysis\RedditNews_preprocessed.csv", index = True)
    print(f"File saved")
except:
    print(f"File saving failed")

```

The preprocessed data is then saved to the csv called, “RedditNews_preprocessed.csv”

Develop a set of words and word combinations that you intend to use to establish positive and negative sentiments.

```
import nltk
from nltk.sentiment import SentimentIntensityAnalyzer
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
```

```
def get_score_nltk(text):
    nltk.download('vader_lexicon', quiet=True)
    sia = SentimentIntensityAnalyzer()
    sentiment_score = sia.polarity_scores(text)
    return sentiment_score["compound"]
```

```
data['Sentiment_score'] = data['Preprocessed Text'].apply(get_score_nltk)
```

```
data
```

A specific sentiment analysis tool called VADER (Valence Aware Dictionary and Sentiment Reasoner) is made to analyze sentiment in textual data; it works especially well with short messages and social media. The VADER Sentiment Analyzer has the following important features:

- VADER makes use of a pre-established vocabulary of terms linked to sentiment values that are neutral, positive, or negative. Because every word in the lexicon has a sentiment score, the system can assess the sentiment of the input text just by looking at the words' presence.
- VADER's accuracy in sentiment evaluation is improved by its ability to handle negations (like "not good") and intensifiers (like "very good").
- In addition to offering scores for positive, negative, and neutral sentiments, it also offers a compound score that, on a range from -1 (most negative) to +1 (most positive), represents the text's overall sentiment.

	Date	News	Preprocessed Text	Sentiment_score
0	Date	News	news	0.0000
1	2016-07-01	A 117-year-old woman in Mexico City finally re...	117 year old woman Mexico City finally receive...	-0.5994
2	2016-07-01	IMF chief backs Athens as permanent Olympic host	IMF chief back Athens permanent olympic host	0.0000
3	2016-07-01	The president of France says if Brexit won, so...	president France say Brexit win Donald Trump	0.5859
4	2016-07-01	British Man Who Must Give Police 24 Hours' Not...	british Man Police 24 hour notice sex threaten...	-0.5187
...	...	...	...	...

This gives the sentiment score for each news.

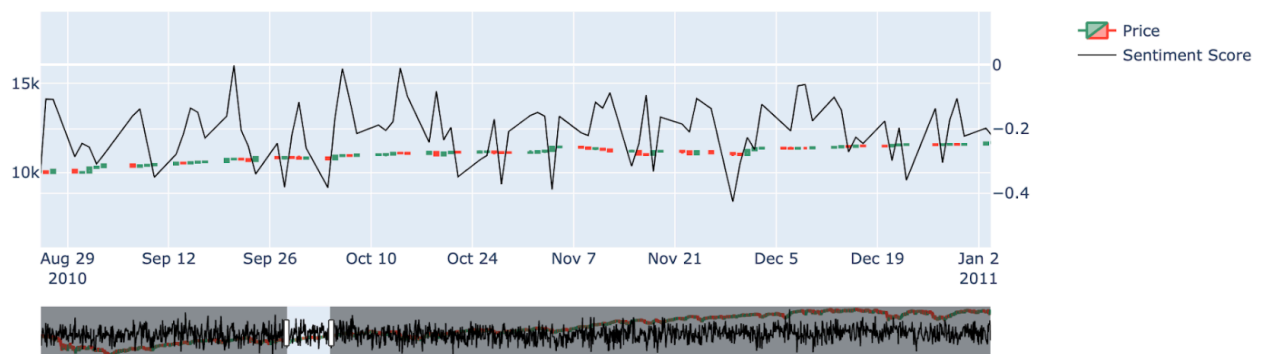
Another csv named, “upload_DJIA_table.csv” contains the opening price, closing price, high,low prices on each date.

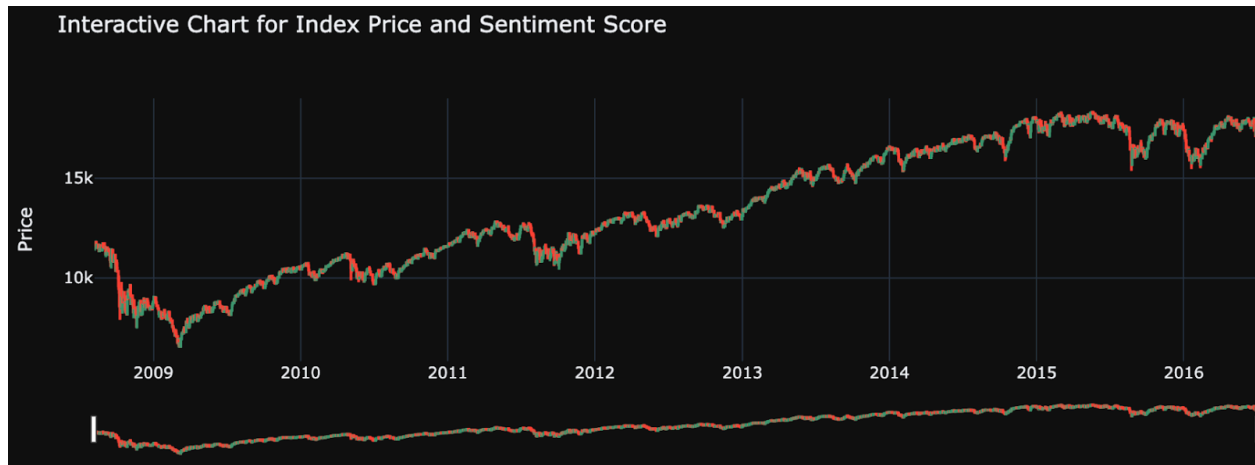
```
2]: index_df
```

	Date	Open	High	Low	Close	Volume	Adj Close
0	2016-07-01	17924.240234	18002.380859	17916.910156	17949.369141	82160000	17949.369141
1	2016-06-30	17712.759766	17930.609375	17711.800781	17929.990234	133030000	17929.990234
2	2016-06-29	17456.019531	17704.509766	17456.019531	17694.679688	106380000	17694.679688
3	2016-06-28	17190.509766	17409.720703	17190.509766	17409.720703	112190000	17409.720703
4	2016-06-27	17355.210938	17355.210938	17063.080078	17140.240234	138740000	17140.240234
...	...	...	...	...	...	...	...

The preprocessed news data and this data are merged, once merged they are grouped based on the average sentimental score of the news in that particular date.

Then the prices against the sentimental scores are analysed.





Phase 5

Deliverables

- Report requirement assessment for each scenario (3)

Steps

- Review the data model you created and compare it to the one provided here. If necessary consider what changes might be needed. You can decide whether you need to change the

underlying database, use views, or use the data modeling and relationship tools in PowerBI to ensure that you are properly representing the available data.

- Review the report scenarios above (3)
- Apply the five step report requirement process to each of the scenarios
- What is the final output of the requested report for each scenario?
- Where am I going to get the data?
- What relationships will I need to define to get that data?
- What are the constraints on the relationships?
- Additional analysis rules (e.g., how we are going to filter, aggregate etc.)
- Develop a report(s) in Power BI to allow your stakeholders to explore this data interactively from a variety of perspectives and to meet the requirements identified in the 3 scenarios

Scenario 1:

Who : Customers

What: assets,state,age,gender

Where: USA

Why: demographic information

How: segments

What is the final output of the requested report for each scenario?

- Five visualizations for each question in scenario 1 plotted on demographic information.

Where am I going to get the data?

- Age_range,customer_id,gender,state,marital_status from **v_eligible_customers**
- Number_of_dependents .total_assets from **v_eligible_customers_dependents**
- Age_range,gender,range_assets,avg_assets from **v_segments**.

What relationships will I need to define to get that data?

- V_eligible_customers

Created view using customer_assets and eligible_customers ec on
ca.customer_id=ec.customer_id where total < 100000 order by
ec.customer_id ;

- V_eligible_customers_dependents

Created view using v_eligible_customers And customer_assets on
cs.customer_id=ec.customer_id ;

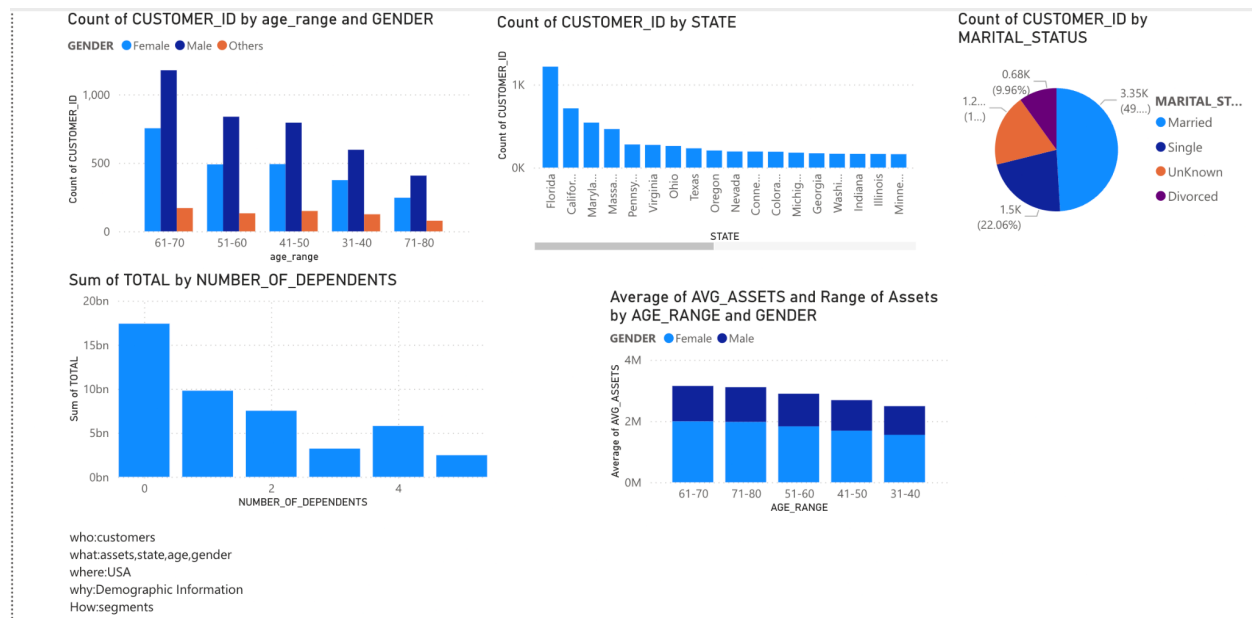
What are the constraints on relationships?

- Inner join

Additional analysis rules (e.g., how we are going to filter, aggregate etc.)

- Averaging assets to get avg_assets for segment analysis.
- Used filter condition for v_eligible_customers having total assets > 100000.

Dashboard:



Scenario 2:

What is the relationship between the risk category and age?

What is the relationship between available assets and risk category?

Are there regional variations in the distribution of risk category?

Who: Customers

What: assets, state, age, risk

Where: USA

Why: Distribution of Risk Profile

How: Segments

1. Identifying Report Elements:

The final output should consists of the report comprising the graphs between age, risk category, customers' minimum,maximum and average assets in each risk profile, and finally the min,max,avg risk for each state.

Risk Profile,state,age,assets(total value)

2. Identifying Report Structures

The v_age_by_risk contains age_range,v invst_by_risk contains contains the details about the assets of the customers, v_risk_by region contains the state and in,max,avg risk in each state

3. Exploring Source Relationships

V_age_by_risk - view created by joining the customers with customer_risk_profile, on condition used is

v_eligible_customers.customer_id=v_customer_risk_profile.customer_id

V_invst_by_risk - view created by joining the customers with customer_risk_profile, on condition used is

v_eligible_customers.customer_id=v_customer_risk_profile.customer_id and then joining with the customer_assets based on the condition

customer_assets.customer_id=v_eligible_customer_risk_profile.customer_id

V_risk_by_region - view created by joining the customers with customer_risk_profile, on condition used is

v_eligible_customers.customer_id=v_customer_risk_profile.customer_id

4. Identifying Relationship Constraints

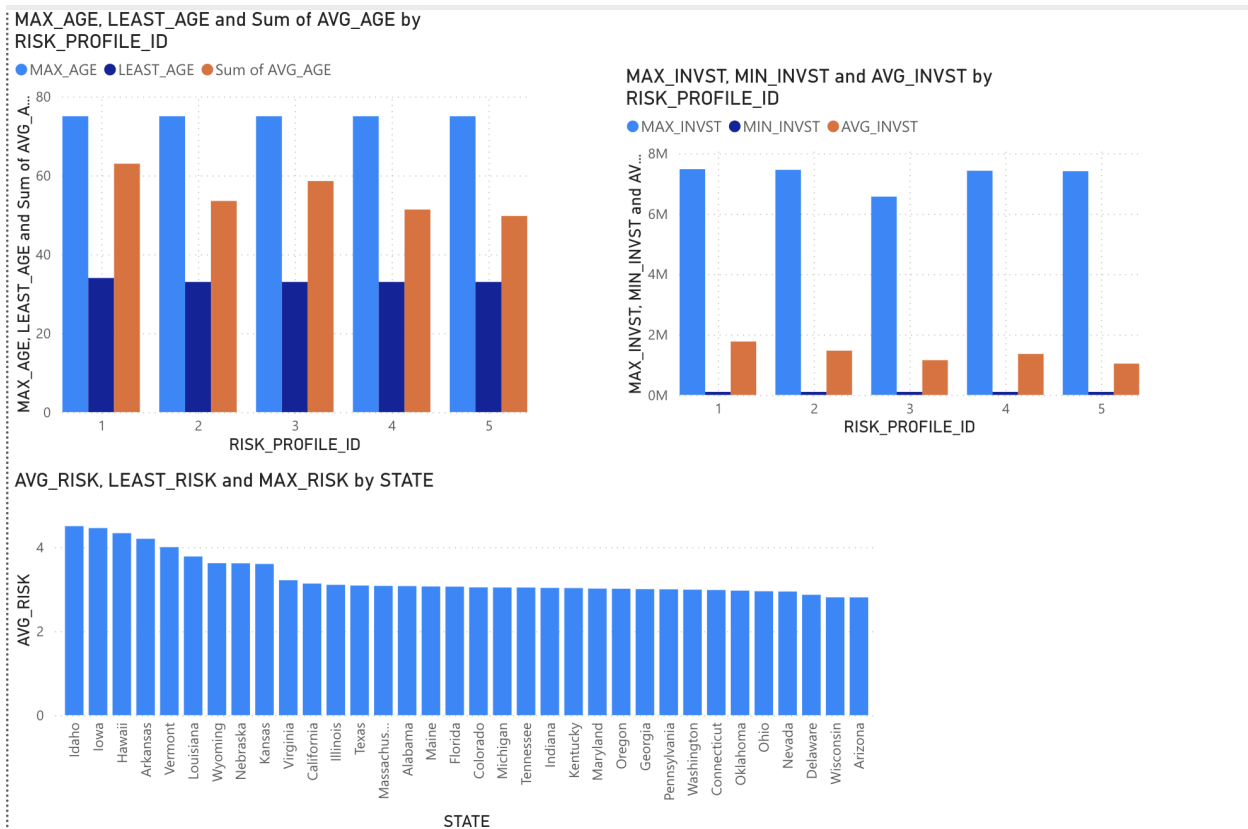
Using inner join operation for joining based on the conditions defined before.

5. Identifying Report Constraints

For question 1 and 2 namely finding the investments for each risk, and age distribution for each risk, the two views are created by grouping using risk_profile_id.

For question 3: risk_by_region it is grouped by using state.

Dashboard:



Scenario 3

Appendix

Appendix 1

View 1

create or replace view

```
CASESTUDY_WORKING.PUBLIC.V_CUSTOMERS_AGE_DISTRIBUTION (AGE_RANGE,  
NO_OF_CUSTOMERS) as select get_age_range(age) as age_range ,count(customer_id) as  
no_of_customers from eligible_customers group by get_age_range(age);
```

View2

Age and Gender Distribution:

create or replace view v_age_gender_dist as select count(customer_id) as
NO_OF_CUSTOMERS,coalesce(gender,'OTHERS') as gender,get_age_range(age) as
AGE_RANGE from v_eligible_customers group by gender,get_age_range(age);

V_REGIONAL_DISTRIBUTION

create or replace view
CASESTUDY_WORKING.PUBLIC.V_REGIONAL_DISTRIBUTION(STATE,
CITY, NO_OF_CUSTOMERS) as select state,city ,count(customer_id) as
no_of_customers from eligible_customers group by city,state order by state;

V_MARITAL_STATUS

create or replace view
CASESTUDY_WORKING.PUBLIC.V_MARITAL_STATUS(
MARITAL_STATUS, NO_OF_CUSTOMERS) as select Coalesce(
marital_status,'Unknown') as marital_status,count(customer_id) as
no_of_customers from eligible_customers group by marital_status;

V_ELIGIBLE_CUSTOMERS

create or replace view
CASESTUDY_WORKING.PUBLIC.ELIGIBLE_CUSTOMERS(CUSTOMER_I
D, CONTACT_LAST_NAME, CONTACT_FIRST_NAME, STREET, CITY,
STATE ,ZIP, COUNTRY,
AGE,MARITAL_STATUS,GENDER,NUMBER_OF_DEPENDENTS, CID,
"COUNT(CA.QUESTION_ID)") as (

with valid_customer_ids as (select customer_id as cid,count(ca.question_id) from
customer_answers ca join answers a on ca.answer_id=a.answer_id and
ca.question_id =a.question_id group by customer_id having
count(ca.question_id)>=6)

select * from customers c inner join valid_customer_ids vc on
c.customer_id=vc.cid);

V_ELIGIBLE_CUSTOMERS_DEPENDENTS

create or replace view

```
CASESTUDY_WORKING.PUBLIC.V_ELIGIBLE_CUSTOMERS_DEPENDEN  
TS( CUSTOMER_ID, NUMBER_OF_DEPENDENTS, TOTAL) as select  
ec.customer_id, ec.number_of_dependents, cs.total from v_eligible_customers ec  
join customer_assets cs on cs.customer_id=ec.customer_id ;
```

V_AGE_GENDER_SEGMENTS

create or replace view

```
CASESTUDY_WORKING.PUBLIC.V_AGE_GENDER_SEGMENTS(  
GENDER, AGE_RANGE, MIN_ASSETS, MAX_ASSETS, SEGMENT_ID,  
RANGE_OF_ASSETS, AVG_ASSETS) as select coalesce(gender,'Others') as  
gender, get_age_range(age) as age_range, min(ca.total) as min_assets,  
max(ca.total) as max_assets,  
get_segment(gender,get_age_range(age)) as segment_id,  
concat(round(min(ca.total),2),' - ',round(max(ca.total),2)) as range_of_assets,  
avg(ca.total) as avg_assets, from v_eligible_customers c join customer_assets ca  
on c.customer_id=ca.customer_id group by gender, get_age_range(age) ,  
get_segment(gender,get_age_range(age))  
having gender !='Others'  
order by segment_id;
```

V_GENDER_SEGMENTS

```
create or replace view v_gender_segments as select segment_id,count(CASE  
WHEN gender='Male' THEN 1 END) as male_count, count(CASE WHEN  
gender='Female' THEN 1 END) as female_count from customer_segment cs join  
v_eligible_customers ec on ec.customer_id=cs.customer_id group by segment_id;
```

v_marital_status_segments

```
create or replace view v_marital_status_segments as select  
segment_id,count(CASE WHEN marital_status='Married' THEN 1 END) as  
married_count, count(CASE WHEN marital_status='Single' THEN 1 END) as  
single_count, count(CASE WHEN marital_status='Divorced' THEN 1 END) as  
divorced_count from customer_segment cs join v_eligible_customers ec on  
ec.customer_id=cs.customer_id group by segment_id;
```

V_AGE_BY_RISK

create or replace view

CASESTUDY_WORKING.PUBLIC.V_AGE_BY_RISK(AVG_AGE,MAX_AGE
,LEAST_AGE,PCT_CST,HEAD_COUNT,RISK_PROFILE_ID) as (

select avg(age) as avg_age,max(age) as max_age, min(age) as
least_age,(count(c.customer_id) / (select count(*) from v_eligible_customers)) *
100 as pct_cst, count(c.customer_id) as head_count , risk_profile_id as
risk_profile_id from v_eligible_customers c join v_customer_risk_profile cr on
c.customer_id = cr.customer_id group by risk_profile_id order by
count(c.customer_id) desc);

V_INVST_BY_RISK

create or replace view

CASESTUDY_WORKING.PUBLIC.V_INVST_BY_RISK(AVG_INVST,
MAX_INVST, MIN_INVST, PCT_CST, HEAD_COUNT, RISK_PROFILE_ID)
as (select sum(total) as avg_invst, max(total) as max_invst, min(total) as
min_invst,(count(c.customer_id) / (select count(*) from v_eligible_customers)) *
20 as pct_cst, count(distinct c.customer_id) as head_count , risk_profile_id as
risk_profile_id from v_eligible_customers c join v_customer_risk_profile cr on
c.customer_id = cr.customer_id join customer_assets ca on cr.customer_id =
ca.customer_id group by risk_profile_id order by count(c.customer_id) desc)